

ONE-STOP DIGITAL RENTAL MARKETPLACE PLATFORM

*A Main Project Report submitted in partial fulfillment of the
requirements for the award of degree of*

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

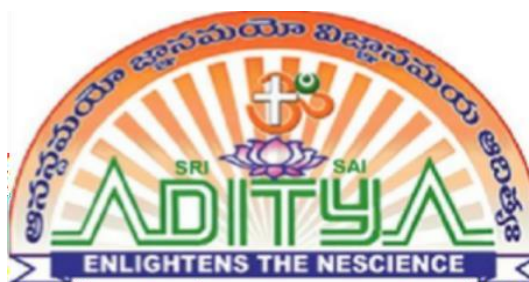
Submitted

By

SANDIP PAL	(22A91A0561)
BACHALA PRUDVI SATYA TEJA	(22A91A0565)
ETTI S N V V SUCHANDRA	(22A91A0570)

Under the esteemed guidance of

Mr. UV Ramesh, M.Tech(Ph.D)
Assistant Professor, CSE



Department of Computer Science and Engineering

ADITYA ENGINEERING COLLEGE

(An Autonomous Institution)

(Approved by AICTE, New Delhi, Affiliated to JNTUK Kakinada, Accredited by NBA and NAAC with 'A++' Grade)

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

2022 – 2026

ADITYA ENGINEERING COLLEGE

(An Autonomous Institution)

(Approved by AICTE, New Delhi, Affiliated to JNTUK Kakinada, Accredited by NBA and NAAC with 'A++' Grade)

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

2022 – 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project work entitled “**ONE-STOP DIGITAL RENTAL MARKETPLACE PLATFORM**” is being submitted by

SANDIP PAL	(22A91A0561)
BACHALA PRUDVI SATYA TEJA	(22A91A0565)
ETTI S N V V SUCHANDRA	(22A91A0570)

in partial fulfillment of the requirements for award of the B.Tech. degree in Computer Science and Engineering.

Project Guide

Mr.UV Ramesh M.Tech(Ph.D)
Assistant Professor
Department of CSE

Head of the Department

Dr.T.SudhaRani,M.Tech,Ph.D
Assistant Professor
Department of CSE

External Examiner

DECLARATION

We hereby declare that the project entitled **“ONE-STOP DIGITAL RENTAL MARKETPLACE PLATFROM”** is a genuine project. This work has been submitted to the **ADITYA UNIVERSITY (Formerly Aditya Engineering College (A))**, Surampalem in partial fulfillment of the **B.Tech.** degree. We further declare that this project work has not been submitted in full or part of the award of any degree of this or any other educational institutions.

By

SANDIP PAL (22A91A0561)

BACHALA PRUDVI SATYA TEJA (22A91A0565)

ETTI S N V V SUCHANDRA (22A91A0570)

ACKNOWLEDGEMENT

It is with immense pleasure that we would like to express our indebted gratitude to my project Guide **Mr. UV Ramesh M.Tech(Ph.D),Assistant Professor, Department of CSE** who has guided us a lot and encourage in every step of the project work, his valuable moral support and guidance throughout the project helped us a greater extent.

We owe our sincere gratitude to our project coordinator **Dr.AppalaRaju Grandhi, M.Tech.Ph.D, Assistant Professor, Department of CSE** for providing a great support and guidance throughout the project.

We are grateful to **Dr.T.Sudha Rani,M.Tech,Ph.D Associate Professor and HOD**, for inspiring us all the way and for arranging all the facilities and resources needed for our project.

We wish to thank our **Dr. M.V. Rajesh, Associate Dean** for their encouragement and support during the course of our project.

I would like to extend my sincere thanks to **Dr. G. Suresh, Registrar; Dr. A. Ramesh, Pro Vice-Chancellor, E&S; Dr. S. Rama Sree, Pro Vice-Chancellor, Academics; Dr. M.B. Srinivas, Vice-Chancellor; Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor, and Management, Aditya University** for unconditional support for me the best infrastructural facilities and state of the art laboratories during my intern project work.

Not to forget, **Faculty, Lab Technicians, Non-Teaching Staff and our Friends** who have directly or indirectly supported us in completing this project on time.



ADITYA ENGINEERING COLLEGE (A)

Aditya Nagar, ADB Road, Surampalem

VISION & MISSION OF THE INSTITUTE

Vision:

To emerge as a premier institute for quality technical education and innovation.

Mission:

M1: Provide learner centric technical education towards academic excellence

M2: Train on technology through collaborations

M3: Promote innovative research & development

M4: Involve industry institute interaction for societal needs

Sd/- 

PRINCIPAL
PRINCIPAL

ADITYA ENGINEERING COLLEGE
SURAMPALAM - 533 437



ADITYA ENGINEERING COLLEGE (A)

Aditya Nagar, ADB Road, Surampalem

Department of Computer Science and Engineering

VISION & MISSION OF THE DEPARTMENT

VISION:

To emerge as a competent Centre of excellence in the field of Computer Science and Engineering for industry and societal needs.

MISSION:

- Impart quality and value based education.
- Inculcate the inter personal skills and professional ethics.
- Enable research through state-of-the-art infrastructure.
- Collaborate with industries, government and professional societies.

A handwritten signature in black ink, appearing to be 'L. R. N.', is written over a circular stamp.

Head of the Department

Head of the Department

Department of CSE

ADITYA ENGINEERING COLLEGE (A)



ADITYA ENGINEERING COLLEGE (A)

Aditya Nagar, ADB Road, Surampalem

Department of Computer Science and Engineering

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

Graduates of the Program will

- **PEO 1:** Adopt new technologies and provide innovative solutions.
- **PEO 2:** Be employable, become an entrepreneur or researcher for a successful career.
- **PEO 3:** Demonstrate interpersonal, multi-disciplinary skills and professional ethics to serve society.

A handwritten signature in red ink, appearing to be 'A. N. R.', is placed above the printed name of the Head of the Department.

Head of the Department

Head of the Department

Department of CSE

ADITYA ENGINEERING COLLEGE (A)



ADITYA ENGINEERING COLLEGE (A)

Aditya Nagar, ADB Road, Surampalem

Department of Computer Science and Engineering

PROGRAM OUTCOMES (POs)

After successful completion of the program, the graduates will be able to

- PO 1 **Engineering Knowledge:** Apply knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
- PO 2 **Problem Analysis:** Identify, formulate, research literature and analyze complex engineering problems, reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.
- PO 3 **Design/Development of Solutions:** Design solutions for complex engineering problems and design systems, components or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations.
- PO 4 **Conduct Investigations of Complex Problems:** Conduct investigations of complex problems using research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.
- PO 5 **Modern Tool Usage:** Create, select and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling, to complex engineering activities, with an understanding of the limitations.
- PO 6 **The Engineer and Society:** Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice.
- PO 7 **Environment and Sustainability:** Understand the impact of professional engineering solutions in societal and environmental contexts and demonstrate knowledge of, and need for sustainable development.

- PO 8 **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
- PO 9 **Individual and Teamwork:** Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.
- PO 10 **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- PO 11 **Project Management and Finance:** Demonstrate knowledge and understanding of engineering management principles and apply these to one's own work, as a member and leader in a team and to manage projects in multidisciplinary environments.
- PO 12 **Life-Long Learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



Head of the Department

Head of the Department

Department of CSE

VIDYA ENGINEERING COLLEGE (A)



ADITYA ENGINEERING COLLEGE (A)

Aditya Nagar, ADB Road, Surampalem

Department of Computer Science and Engineering

PROGRAM SPECIFIC OUTCOMES (PSOs)

After successful completion of the program, the graduates will be able to

PSO 1: Develop efficient solutions to real world problems using the domains of Algorithms, Networks, database management and latest programming tools and techniques.

PSO 2: Provide data centric business solutions through emerging areas like IoT, AI, data analytics and Block Chain technologies.

Head of the Department

Head of the Department

Department of CSE

ADITYA ENGINEERING COLLEGE (A)

ABSTRACT

The rapid growth of the sharing economy has significantly increased the demand for rental services as a cost-effective and flexible alternative to ownership. However, most existing rental platforms are concentrated in metropolitan areas, making it difficult for users in Tier-2 and Tier-3 cities to access reliable rental services. At the same time, local vendors in these regions face challenges such as limited digital presence, low customer reach, and high marketing costs. This project presents a One-Stop Digital Rental Marketplace Platform designed to bridge this gap by connecting users with verified local vendors and individual item owners through a unified digital system. The platform supports multiple rental categories including cars, bikes, cameras, laptops, mobile phones, furniture, home appliances, gaming consoles, tools, musical instruments, event equipment, sports gear, drones, clothing, and more, providing users with a convenient and centralized solution. The system is built using modern web technologies including Next.js with TypeScript for the frontend, Supabase PostgreSQL for the backend database and authentication, Tailwind CSS for responsive styling, and Vercel for cloud deployment. Users can browse listings through an Airbnb-inspired interface, search and filter by category, city, and price range, and contact owners directly via phone or WhatsApp. This platform enhances accessibility, improves transparency, and creates a seamless rental discovery experience for users across all city tiers. It operates as a pure listing and discovery platform where no payment processing occurs on the platform itself, allowing renters and owners to connect directly and handle rental transactions privately between themselves. Additionally, it reduces dependency on commission-based intermediaries and empowers local vendors with digital visibility and analytics. The proposed system has potential applications across diverse rental markets, small business empowerment, and digital marketplace ecosystems. Future enhancements may include native mobile applications, payment gateway integration, AI-powered item recommendations, map-based search, and multi-language support. Overall, this project demonstrates an effective way to combine modern web development technologies with marketplace design principles to create an intelligent, scalable, and user-centric rental platform that addresses real-world challenges within the sharing economy.

INDEX

LIST OF CONTENTS

CONTENTS	PAGE NO
Chapter 1: INTRODUCTION	
1.1 Introduction to Digital Rental Marketplace Systems	1-2
1.2 Existing System and Disadvantages	3
1.3 Proposed System and Advantages	4
1.4 Objective of the Project	5
1.5 Organization of the Project	6
 Chapter 2: REQUIREMENT ANALYSIS	
2.1 Introduction	8
2.2 Hardware & Software Requirements	9-10
2.3 System Requirements Specification	11-13
 Chapter 3: LITERATURE SURVEY	
3.1 Related Works	14-15
3.2 Survey Table	16
 Chapter 4: MODULE	
4.1 Introduction to Modules	17-19
4.2 Module Implementation	20-22
 Chapter 5: SYSTEM DESIGN	
5.1 Introduction to System Design	23
5.2 Process Flow of Project	24-25
5.3 Flowchart Diagram	26
5.4 Use case Diagram	27-28
5.5 Sequence Diagram	29
5.6 Activity Diagram	30
5.7 Class Diagram	31-32
5.8 State Diagram	33-34
5.9 Component Diagram	35-36

Chapter 6: SYSTEM IMPLEMENTATION

6.1 Introduction to System Implementation	37
6.2 Selection of Software	37
6.3 Programming Languages	38
6.4 Backend-as-a-Service	39
6.5 Database & Storage Solutions	40
6.6 Authentication & Security System	41
6.7 User Interface Development	42
6.8 Sample Code	43-44

Chapter 7: TESTING

7.1 Introduction	45
7.2 Test cases	46
7.3 Unit Testing	47
7.4 Integration Testing	48
7.5 System Testing	49
7.6 User Acceptance Testing	50
7.7 Performance Testing	51

Chapter 8: SCREENS & REPORTS

8.1 Introduction	52
8.2 Screenshots	53
8.3 Home Page & Category Bar Screen	54
8.4 Authentication & Login Modal Screen	55
8.5 Listing Detail & Contact Owner Screen	56
8.6 Create Listing & Multi-Step Form Screen	57
8.7 Search, Filter & Wishlist Screen	58

Chapter 9: CONCLUSION AND FUTURE SCOPE

9.1 Conclusion	59-60
9.2 Future Scope	61-62

Chapter 10: BIBLIOGRAPHY

10.1 Weblinks	63-64
10.2 References	65

	LIST OF TABLES	
TABLE.NO	LIST OF TABLES	PG.NO
2.2	HARDWARE REQUIREMENTS	9
2.2	SOFTWARE REQUIREMENTS	11
2.3	SYSTEM REQUIREMENTS SPECIFICATION	14
3	LITERATURE SURVEY	16
4	MODULES DESCRIPTION	19
6.5	DATABASE TABLES DESCRIPTION	45
7.2	UNIT TESTING RESULTS	51
7.4	INTEGRATION TESTING RESULTS	53
7.5	SYSTEM TESTING RESULTS	54
7.6	USER ACCEPTANCE TESTING	55
7.7	PERFORMANCE TESTING RESULTS	56

LIST OF FIGURES

FIGURE.NO	NAME OF THE FIGURES	PG.NO
5.1	SYSTEM ARCHITECTURE DIAGRAM	23
5.2	PROCESS FLOW OF RENTAL MARKETPLACE SYSTEM	25
5.3	FLOWCHART	27
5.4	USECASE	29
5.5	SEQUENCE	31
5.6	ACTIVITY	33
5.7	CLASS	36
5.8	STATE	38
5.9	COMPONENT	40
8.1	HOME PAGE INTERFACE	57
8.2	AUTHENTICATION MODAL	58
8.3	CATEGORY BAR & LISTING GRID	59
8.4	LISTING DETAIL PAGE	60
8.5	CONTACT OWNER MODAL	61
8.6	CREATE LISTING FORM	62
8.7	SEARCH & WISHLIST SCREEN	63

1.INTRODUCTION

The sharing economy has emerged as a transformative force in the global marketplace, fundamentally changing how people access goods and services. Instead of purchasing items outright, consumers increasingly prefer to rent products temporarily, driven by cost efficiency, environmental consciousness, and the flexibility that rental arrangements provide. With the exponential growth of digital platforms and online marketplace services, users now have access to a wide range of products available for rent across different categories, locations, and price ranges. While this abundance offers convenience and variety, it also creates significant challenges in discovering reliable rental services, especially in regions where organized digital platforms are largely absent.

Rental markets in India have traditionally been fragmented and localized, with users relying primarily on word-of-mouth recommendations, local classified advertisements, and neighborhood shops to find items for rent. These informal approaches lack transparency, standardized pricing, trust mechanisms, and the convenience that modern digital platforms can provide. Therefore, there is a growing need for a unified digital rental marketplace that can organize these fragmented markets and connect renters with item owners efficiently.

The proposed project addresses this challenge by developing a comprehensive One-Stop Digital Rental Marketplace Platform that enables users to browse, search, and discover rental items across multiple categories through a single, intuitive web application. The platform uses modern web technologies to provide an Airbnb-inspired user interface, secure authentication, advanced search and filtering, review and rating systems, and direct contact capabilities via phone and WhatsApp. This approach enhances accessibility, reduces the effort required to find rental items, and improves user satisfaction by providing a transparent and vendor-friendly solution.

1.1 INTRODUCTION TO DIGITAL RENTAL MARKETPLACE SYSTEMS

Digital rental marketplace system is an advanced web-based platform designed to connect users seeking rental items with owners or vendors who have products available for rent. With the rapid expansion of the sharing economy and the increasing penetration of internet and smartphone usage across India, the demand for organized rental services has grown substantially. Marketplace platforms address this demand on by providing centralized systems in where users can able to browse available items and also they can compare options, evaluate owner credibility, and can also able to initiate contact for rental arrangements with others.

In general, digital rental marketplace systems operate by allowing item owners to create detailed listings that include photographs, descriptions, pricing information, location details, availability status, and contact information. Renters can then search for items using keywords, category filters, location-based search, and price range filters. The platform facilitates the discovery and connection process, enabling both parties to communicate and arrange the rental transaction.

There are several approaches used in existing rental marketplace platforms

- **Single-Category Platforms:**

These platforms focus on one specific product category such as vehicles, furniture, or electronics. Examples include Zoomcar for cars, Rentomojo for furniture, and Royal Brothers for two-wheelers. While effective within their niche, they require users to visit multiple platforms for different rental needs.

- **General Classifieds Platforms:**

Platforms like OLX and Quikr allow users to post both sale and rental listings, but they lack specialized rental features such as rental period pricing, availability tracking, and category-specific search filters.

- **Commission-Based Marketplace Platforms:**

Many existing platforms charge vendors a significant commission on each transaction, typically ranging from fifteen to thirty percent. This model discourages small vendors and individual owners from participating in the platform.

- **Peer-to-Peer Rental Platforms:**

Despite these approaches, existing rental marketplace systems have significant limitations. Most platforms are concentrated in metropolitan cities, leaving users in Tier-2 and Tier-3 cities without access to organized rental services. They typically support only a single product category, charge high commissions, and lack comprehensive trust mechanisms such as user verification, reviews, and ratings.

To overcome these limitations, the proposed platform provides a unified, multi-category, commission-free rental marketplace that serves users across all city tiers. The system supports sixteen distinct rental categories including cars, bikes, cameras, laptops, mobile phones, furniture, home appliances, gaming consoles, tools and equipment, sports gear, musical instruments, event equipment, books, drones, clothing and costumes, and others. Users can create listings through a guided seven-step process, search for items using advanced filters, save items to their wishlist, leave reviews, and contact owners directly.

The platform operates as a listing and discovery system where actual rental transactions occur directly between the renter and owner through phone calls or WhatsApp messages. For example:

- Cars and Bikes → Contact owner for pickup arrangements
- Cameras and Laptops → Verify item condition before rental
- Furniture and Appliances → Arrange delivery or self-pickup
- Gaming and Sports Equipment → Agree on rental duration and deposit

This direct contact approach simplifies the platform architecture, eliminates payment complexity, and provides a familiar experience similar to how rental transactions traditionally occur in India.

1.2 EXISTING SYSTEM AND DISADVANTAGES

Existing Existing rental services in India are provided through a combination of offline shops, single-category digital platforms, and general classifieds websites. These systems have served the market to varying degrees but face numerous limitations that restrict their reach, effectiveness, and user satisfaction.

In the offline rental market, users discover available items through personal networks, local advertisements, or physical visits to rental shops. This approach severely limits the selection of available items, provides no mechanism for comparing prices or evaluating vendor reliability, and requires significant time and effort from the user. In the digital space, platforms like Zoomcar specialize in car rentals, Rentomojo focuses on furniture and appliances, and Royal Brothers offers two-wheeler rentals. While these platforms have improved the rental experience within their respective categories, operate exclusively in metropolitan cities & charge substantial commissions to vendors.

Disadvantages of Existing Systems:

- **Single-Category Focus::**

Most existing platforms serve only one product category. A user who needs to rent a car, a camera, and furniture must navigate three completely different platforms with separate registrations, interfaces, and trust mechanisms. This fragmented experience is inefficient and time-consuming.

- **Limited Geographic Coverage:**

Organized rental platforms are overwhelmingly concentrated in Tier-1 cities such as Bangalore, Mumbai, Delhi, and Hyderabad. Users in Tier-2 and Tier-3 cities such as Vizag, Jaipur, Kochi, Lucknow, and Chandigarh have virtually no access to digital rental services.

- **High Commission Charges:**

Most platforms charge vendors between fifteen to thirty-five percent of each transaction as commission. This reduces vendor profitability, increases rental prices for users, and creates barriers for small-scale vendors and individual item owners

- **Poor Vendor Support:**

Existing platforms are designed primarily for business vendors and offer limited tools for individual owners who want to rent out idle personal items. The onboarding processes and operational requirements are typically complex and not suitable for casual owners.

- **Limited Trust Mechanisms:**

Comprehensive verification systems, detailed review mechanisms, and transparent rental histories are often absent or poorly implemented in existing platforms.

1.3 PROPOSED SYSTEM AND ADVANTAGES

The proposed system introduces a comprehensive and unified digital rental marketplace platform that connects users seeking rental items with verified owners and vendors across multiple categories. Unlike traditional systems that depend on single categories and commission-based models, this platform focuses on providing a transparent, accessible, and vendor-friendly solution.

The system is designed using modern web technologies including Next.js with TypeScript for the frontend framework, Supabase with PostgreSQL for the backend database and authentication services, Tailwind CSS for responsive styling, and Vercel for cloud deployment. The platform features an Airbnb-inspired user interface that provides a familiar and intuitive browsing experience.

Working Process of the Proposed System:

- User registers or logs in using email and password or Google OAuth
- Browse rental listings through the responsive card-based home page grid
- Filter listings by category using the horizontal scrolling category bar
- Search for items by keyword, city, price range, and condition
- View detailed listing pages with photos, pricing, owner information, and reviews
- Contact the owner directly via phone or WhatsApp through the contact modal
- Create new listings through a guided seven-step form with photo upload
- Manage listings, view analytics, and track performance through the owner dashboard

The system continuously updates available listings and provides real-time search results, ensuring that users always have access to the most current rental options in their area.

Advantages of the Proposed System:

- **Unified Multi-Category Platform::**
Supports sixteen rental categories under a single interface, eliminating the need to visit multiple platforms for different rental needs.
- **Commission-Free Model: :**
Designed to serve users and vendors across all city tiers, with special emphasis on Tier-2 and Tier-3 cities where organized platforms are currently absent.
- **Geographic Inclusivity::**
Eliminates the need for manual searching, selection, or filtering of songs.
- **Individual Owner Empowerment:**
Any individual can list items for rent through a simple guided process, not just established businesses, democratizing the rental marketplace.
- **Comprehensive Trust Mechanisms::**
Provides User verification badges, star ratings, written reviews, and pricing.

1.4 OBJECTIVES OF THE PROJECT

The primary objective of this project is to design and develop a unified multi-category digital rental marketplace platform that connects users seeking rental items with verified owners and vendors, simplifying the rental discovery process while providing transparency, accessibility, and trust. The system aims to enhance user experience by providing a modern, responsive, and feature-rich web application without requiring payment processing on the platform.

Specific Objectives:

- **To develop a multi-category rental listing system::**
Implement support for sixteen distinct rental categories with category-specific icons, filtering, and search capabilities to provide a comprehensive one-stop rental solution.
- **To implement secure user authentication:**
Build a robust authentication system supporting email and password registration, Google OAuth login, session management, and automatic profile creation using Supabase Auth.
- **To design an Airbnb-inspired user interface:**
Develop a guided multi-step form using Zustand state management that enables item owners to create comprehensive listings with photos, descriptions, pricing, location, and contact information.
- **To implement advanced search and filtering:**
Provide full-text search, category filtering, city-based search, price range filtering, condition filtering, and multiple sorting options to help users find relevant rental items quickly.
- **To create a review and rating system:**
Build a comprehensive feedback system with star ratings and written reviews that automatically calculates average ratings for listings and owner profiles to establish trust and credibility..
- **To ensure system automation:**
Implement contact functionality that displays phone numbers, generates WhatsApp links with pre-filled messages, and provides email contact options for seamless communication between renters and owners.
- **To provide vendor analytics and management tools:**
Develop an owner dashboard with listing management, status toggling, view and inquiry tracking, and performance metrics to empower vendors with actionable insights.
- **To ensure production-grade deployment:**
Optimize and Deploy the application on Vercel with proper environment configuration, SSL certification, custom domain support, and optimized performance for real-world usage.

1.5 ORGANIZATION OF PROJECT

The project is structured into multiple chapters to ensure systematic documentation and easy understanding:

- **Chapter1: Introduction**

Provides an overview of the project, including background on digital rental marketplaces, analysis of existing systems, description of the proposed system, project objectives, and document organization.

- **Chapter 2: Requirements Analysis**

Describes the hardware and software requirements needed for development and deployment, including system specifications, functional requirements, non-functional requirements, and technology stack details.analysis.

- **Chapter 3: Literature Survey**

Reviews existing research work & commercial platforms related to rentals, sharing economy systems, and digital marketplace, identifying the research gap addressed by this project.

- **Chapter 4: Modules**

Explains the different functional modules of the system such as authentication, listing management, search and discovery, review system, wishlist & DM, and owner dashboard.

- **Chapter 5: System Design**

Includes system architecture design, process flow diagrams, flowcharts, and UML diagrams including use case, sequence, activity, class, state, and component diagrams that illustrate the structural and behavioral aspects of the platform.

- **Chapter 6: System Implementation**

Details the technology selection rationale, programming languages and frameworks used, database and storage configuration, authentication implementation, user interface , also sample code demonstrating key system components.

- **Chapter 7: Testing**

Describes various testing methods including unit testing, integration testing, system testing, user acceptance testing, and performance testing, with detailed test cases and results for all major system modules.

- **Chapter 8: Screens and Reports**

Screens and Reports Presents system screenshots showing the home page, authentication modals,contact owner modals, create listing forms, search and filter, and wishlist pages.

- **Chapter 9: Conclusion and Future Scope**

Summarizes the project outcomes, discusses the significance of the work, and identifies potential future enhancements including mobile app, payment integration, & recommendations.

- **Chapter 10: Bibliography**

Bibliography Lists all references, research papers, web resources, and technical documentation used during the development of this project.

2. REQUIREMENT ANALYSIS

2.1 INTRODUCTION

This section provides a detailed overview of the requirements necessary for developing the One-Stop Digital Rental Marketplace Platform. It serves as a bridge between the conceptual idea of the project and its practical implementation by identifying all essential resources, tools, and system functionalities.

The proposed system combines multiple technologies such as frontend web development, backend-as-a-service, cloud database management, authentication services, and responsive user interface design. Therefore, it requires careful planning to ensure that all components function together efficiently. The introduction to requirements analysis highlights the importance of defining system scope, understanding user needs, and ensuring that the system meets performance expectations.

Purpose of this Section

The main purpose of this section is to:

- Define the overall requirements of the system
- Identify hardware and software components needed
- Describe how different modules interact
- Ensure clarity before system design and implementation

This helps developers and stakeholders understand what the system is expected to achieve.

Scope of the System

The scope of the project includes:

- Allowing users to register and authenticate using email or Google OAuth
- Enabling item owners to create detailed rental listings with photos and pricing
- Providing a responsive Airbnb-inspired interface for browsing rental items
- Supporting advanced search and filtering by category, city, and price range
- Facilitating direct contact between renters and owners via phone and WhatsApp
- Implementing review and rating systems for building trust and transparency

The system is designed to work as a standalone desktop application but can be extended to mobile or web-based platforms in the future.

Overview of System Functionality

The system performs the following key operations:

1. **User Authentication:** Allows users to register and log in using email or Google OAuth
2. **Listing Management:** Enables owners to create, edit, and manage rental listings
3. **Search and Discovery:** Provides category browsing, text search, and advanced filtering
4. **Listing Detail View:** Displays comprehensive item information with images and reviews
5. **Contact and Communication:** Facilitates direct contact via phone, WhatsApp, and email
6. **Review and Rating:** Allows users to submit star ratings and written reviews
7. **Wishlist Management:** Enables users to save and organize favorite listings
8. **Owner Dashboard:** Provides analytics and listing management tools for item owners

Importance of Requirement Introduction

This section ensures that:

- All system needs are clearly identified
- Developers have a clear understanding of implementation requirements
- Potential challenges are recognized early
- The system design phase can proceed smoothly

Overall, this introduction lays the foundation for building an efficient, accurate, and user-friendly emotion-based music recommendation system.

2.2 HARDWARE & SOFTWARE REQUIREMENTS

This section specifies the hardware and software resources required to design, develop, and deploy the One-Stop Digital Rental Marketplace Platform. Proper selection of these requirements ensures smooth system performance, responsive user interface rendering, and efficient handling of database operations and authentication services.

Hardware Requirements

The hardware components form the physical foundation of the system. Since the project is a cloud-deployed web application that leverages Supabase for backend services and Vercel for hosting, the hardware requirements are primarily focused on the development environment. The production deployment utilizes cloud infrastructure that eliminates the need for dedicated server hardware.

Minimum Hardware Requirements:

- **Processor:** Intel Core i3 or equivalent
- **RAM:** 4 GB
- **Storage:** 256 GB HDD or SSD
- **Webcam:** Basic webcam for capturing product images
- **Display:** Standard monitor
- **Input Devices:** Keyboard and mouse

Recommended Hardware Requirements:

- **Processor:** Intel Core i5/i7 or higher
- **RAM:** 8 GB or more
- **Storage:** SSD for faster data access
- **Webcam:** High-resolution camera for better accuracy
- **GPU (Optional):** NVIDIA GPU for faster deep learning processing
- **Network:** Broadband internet connection with minimum 10 Mbps speed

Hardware Description:

- The processor handles computations for running the development server, TypeScript compilation, and hot module replacement during development.
- The RAM supports smooth execution of the Next.js development server, code editor, browser testing, and database management tools simultaneously.
- The SSD storage significantly improves development server startup time, module resolution speed, and overall development workflow efficiency.
- The display with adequate resolution is essential for testing responsive design across multiple viewport sizes and ensuring proper layout rendering.
- The internet connection is required for accessing Supabase cloud services, downloading npm packages, deploying to Vercel, and testing the live application.
- Software Requirements

Software Requirements

Software components include programming tools, libraries, and platforms required to build and execute the system.

Operating System:

- Windows 10 or 11, macOS, or Linux (Ubuntu 20.04 or higher)

Programming Language:

- TypeScript with Next.js 14 or higher (React-based framework with App Router)

Development Tools / IDEs:

- Visual Studio Code with TypeScript and Tailwind CSS extensions
- Anti-Gravity IDE for AI-assisted development
- Git for version control with GitHub for repository hosting
- npm (Node Package Manager) version 9 or higher

Libraries and Frameworks:

- **Tailwind CSS:** For utility-first responsive styling and custom theme configuration
- **Framer Motion:** For smooth page transitions and staggered animations
- **Lucide React:** For consistent and customizable SVG icon components
- **Zustand:** For lightweight state management in multi-step listing creation form
- **react-hot-toast:** For beautiful toast notification components
- **react-dropzone:** For drag-and-drop file upload functionality in listing creation

Database:

- Supabase: Backend-as-a-Service providing PostgreSQL database, Auth, and Storage
- @supabase/ssr: Server-side rendering helpers for Next.js integration
- @supabase/supabase-js: JavaScript client library for database queries

System Configuration Requirements

- Node.js version 18 or higher installed on the development machine
- npm package manager properly configured for dependency management
- Git version control system installed for source code management
- Modern web browser such as Chrome, Firefox, Safari, or Edge for testing
- Supabase account with project configured for database and authentication
- Vercel account connected to GitHub repository for deployment

Additional Considerations

- Environment variables must be properly configured in a .env.local file for Supabase credentials
- The Next.js development server runs on localhost port 3000 by default during development
- Regular updates of npm packages ensure better performance, security, and compatibility

Hardware Requirements

Category	Component	Specification
Processor	CPU	Intel Core i3 or higher
Memory	RAM	Minimum 4 GB (8 GB recommended)
Storage	Disk	500 GB HDD / SSD (SSD preferred)
Input Device	Webcam	Basic (HD recommended)
Output Device	Display	Standard Monitor
Input Devices	Keyboard/Mouse	Required
Optional	GPU	NVIDIA GPU for faster processing

Software Requirements

Category	Tools/Technologies
Programming Language	TypeScript
IDEs	VS Code, Googles-Antigravity
Framework	Next.js with App RouterStylingTailwind CSS
StateManagement	Zustand
Deployment	Vercel Deployer
Version Control	Git with GitHub
Backend & Database	Supabase (PostgreSQL, Auth, Storage)

2.3 SYSTEM REQUIREMENTS SPECIFICATION

The System Requirements Specification (SRS) is a comprehensive description of the system's expected functionality, performance, and constraints. It serves as a formal document that guides the design, development, and testing phases of the project. For the One-Stop Digital Rental Marketplace Platform, the SRS defines how the system should behave and the standards it must meet.

Purpose of SRS

The main purpose of the SRS is to:

- Provide a clear understanding of system functionalities
- Act as a reference for developers, testers, and stakeholders
- Ensure that the final system meets user expectations
- Reduce ambiguity and development errors

Functional Requirements

These requirements describe the core operations that the system must perform:

- These requirements describe the core operations that the system must perform:
- The system should allow users to register using email and password or Google OAuth
- The system should authenticate users and maintain session state across page navigations
- The system should automatically create user profiles upon registration through database triggers
- The system should allow authenticated users to create listings through a seven-step guided form
- The system should support photo upload to cloud storage during listing creation
- The system should display rental listings in a responsive card-based grid with image carousels
- The system should provide a horizontally scrollable category bar with sixteen category icons
- The system should support filtering listings by category, city, price range, and condition
- The system should support full-text search across listing titles, descriptions, and locations
- The system should display detailed listing pages with photos, pricing, owner information, and reviews
- The system should provide a contact modal showing phone, WhatsApp, and email contact options
- The system should generate WhatsApp links with pre-filled messages for owner contact
- The system should allow authenticated users to submit star ratings and written reviews
- The system should automatically calculate and update average ratings for listings and owners
- The system should allow authenticated users to save listings to a personal wishlist
- The system should provide an owner dashboard with listing management and analytics
- The system should support listing status management including active, rented, and inactive states
- The system should provide professional pages including About, How It Works, FAQ, Contact, Terms, and Privacy

Non-Functional Requirements

These requirements define the quality and performance of the system:

- **Performance:**
Pages should load within three seconds on standard broadband connections.
The production build should compile with zero TypeScript or ESLint errors.
- **Accuracy:**
Search results should accurately match user queries. Category filtering should correctly display only items from the selected category.
- **Usability:**
The user interface should follow Airbnb-style design patterns for familiarity and intuitiveness. All interactive elements should have appropriate visual feedback.
- **Reliability:**
The system should function without crashes or interruptions. Error boundaries & should catch and handle rendering errors gracefully.
- **Scalability:**
The database schema and application architecture should support growing numbers of users, listings, and transactions without requiring fundamental architectural changes
- **Maintainability:**
The codebase should be organized into modular components with clear separation of concerns. TypeScript types should be defined for all data structures.
- **Portability:**
The system should work consistently across Chrome, Firefox, Safari, and Edge browsers on desktop, tablet, and mobile devices.
- **Security:**
Row Level Security policies should protect all database tables. Authentication tokens should be refreshed automatically through middleware. User inputs should be validated before database insertion.

System Constraints

- Requires a modern web browser with JavaScript enabled for full functionality
- Performance depends on internet connection speed for database queries and image loading
- Image upload size is limited by Supabase Storage tier constraints
- Authentication rate limits apply based on Supabase free tier allowances
- The platform does not process any kind of the payment department and relies on direct renter-owner communication

Assumptions and Dependencies

Assumptions:

- Users have access to a modern web browser on desktop or mobile devices
- Users have a stable internet connection for browsing and creating listings
- Item owners have phone numbers or WhatsApp for renter contact
- The Supabase free tier provides sufficient resources for initial deployment

Dependencies:

- Supabase cloud services must be available and properly configured for database, authentication, and storage operations
- Vercel deployment platform must be accessible for hosting and continuous deployment
- npm packages including Next.js, Tailwind CSS, and Supabase client libraries must be properly installed
- Google Cloud Console must be configured for OAuth authentication credentials

User Interface Requirements

- The system should display a sticky navigation bar with logo, search bar, and user menu
- The system should show a horizontally scrollable category bar with icons below the navbar
- Listing cards should display item images, titles, locations, prices, and ratings
- The listing detail page should show a comprehensive view with images, pricing & reviews
- The contact modal should present phone, WhatsApp, and email options clearly
- The create listing form should guide users through seven steps with progress indication

Performance Requirements

- Server-side rendering should provide fast initial page loads
- Database queries should use proper indexes for efficient data retrieval
- Images should be optimized for web delivery with appropriate dimensions
- Skeleton loading states should provide visual feedback during data fetching
- Infinite scroll pagination should load additional listings without full page refresh

System Requirements Specification (SRS)

Type	Details
Functional Requirements	User authentication, Listing creation, Search and filtering, Category browsing, Contact owner, Reviews and ratings, Wishlist, Owner dashboard
Non-Functional Requirements	Performance, Reliability, Scalability, Security, Usability, Maintainability, Portability
Constraints	Browser requirement, Internet dependency, Storage limits, Authentication limits, No payment process

3. LITERATURE SURVEY

3.1 RELATED WORKS

In recent years, significant research and commercial development has been carried out in the areas of digital rental marketplaces, peer-to-peer sharing economy platforms, and online marketplace system design. Various approaches, business models, and technologies have been proposed to enhance user experience by providing organized, transparent, and accessible rental services. This section reviews some of the important contributions and approaches related to the proposed system.

Traditional Music Recommendation Systems

Early developments in digital rental marketplaces focused on category-specific approaches:

- **Vehicle Rental Platforms:**

These Platforms such as Zoomcar and Royal Brothers emerged to address the self-drive car and two-wheeler rental market in India. These platforms operate on commission-based models and are concentrated in Tier-1 metropolitan cities. While effective within their niche, they serve only a single product category and are inaccessible to users in smaller cities.

- **Furniture and Appliance Rental Platforms:**

Companies like Rentomojo and Furlenco developed subscription-based models for furniture and home appliance rentals. These platforms maintain their own inventory and deliver items to customers, but their operations are limited to select metropolitan areas due to the logistics.

- **General Classifieds Platforms:**

OLX and Quikr allow users to post listings for both sales and rentals. However, these platforms lack specialized rental features such as rental period pricing, availability calendars, category-specific search filters, and structured review systems.

Peer-to-Peer Sharing Economy Research

Academic researchers have explored the dynamics of peer-to-peer rental markets extensively:

- **Economic Impact Studies:**

Research by Fraiberger and Sundararajan published through Harvard Business School examined how digital platforms facilitate the rental of durable goods between individuals, demonstrating that such platforms increase welfare by improving asset utilization and reducing the need for ownership.

- **Product Category Analysis::**

Studies by Hirao examined which product categories are most suitable for peer-to-peer rental markets, finding that items with high purchase prices, low depreciation rates, and intermittent usage patterns are most conducive to successful rental marketplaces.

- **Trust and Verification Research:**

Multiple studies have mentioned the critical role of trust mechanisms including user verification, review systems, and transparent pricing in the success of peer-to-peer marketplace platforms.

Modern Web Application Development Approaches

Recent advancements in web technologies have significantly improved the capabilities of marketplace platforms:

- **Server-Side Rendering Frameworks:**

Frameworks like Next.js provide server-side rendering capabilities that improve page load performance, search engine optimization, and user experience compared to traditional client-side rendered applications.

- **Backend-as-a-Service Platforms:**

Services like Supabase and Firebase provide complete backend infrastructure including databases, authentication, and file storage, enabling rapid development without custom server implementation.

- **Responsive Design Systems:**

CSS frameworks like Tailwind CSS enable utility-first styling approaches that facilitate rapid development of responsive, mobile-friendly interfaces.

Marketplace Design Pattern Research

Research into marketplace design has identified key patterns for successful platforms:

- **Two-Sided Marketplace Dynamics:** Studies have explored how platforms must balance the needs of both supply-side participants (item owners) and demand-side participants (renters) to achieve sustainable growth.
- **Commission-Free Models:** Research on platforms like NoBroker has demonstrated that zero-commission models can disrupt traditional intermediary-based markets by removing financial barriers for service providers.
- **Geographic Expansion Strategies:** Academic papers have analyzed how marketplace platforms can expand from metropolitan areas to smaller cities, identifying challenges related to supply density, user adoption, and trust establishment.

Integrated Multi-Category Systems

Some recent commercial and research efforts have attempted to create unified multi-category platforms:

- Platforms that combine multiple rental categories under a single interface
- Systems that use location-based search to connect nearby renters and owners
- Applications that integrate review systems with verification badges for trust building

However, many of these systems are still limited in terms of geographic coverage, vendor support, or the comprehensiveness of their feature sets.

Limitations of Existing Related Works

- Lack of unified multi-category rental platforms serving all city tiers in India
- Heavy dependence on commission-based revenue models that discourage small vendors
- Limited support for individual item owners who want to monetize idle personal assets
- Poor integration between search, filtering, trust mechanisms, and direct contact capabilities
- Absence of comprehensive vendor dashboards and analytics tools in most platforms

3.2 SURVEY TABLE

The survey table summarizes different approaches, platforms, and technologies used in existing systems related to digital rental marketplaces and sharing economy platforms. It compares their focus areas, key contributions, technologies used, and notable outcomes. This analysis helps in selecting appropriate methods and design patterns for the proposed system, ensuring better accessibility, transparency, and user experience.

Author(s)	Study Focus	Key Contributions	Technologies Used	Notable Outcomes
Fraiberger, & Sundararajan, (2016)	Peer-to-peer rental markets in the sharing economy	dynamics of P2P rental platforms and demonstrated welfare gains from improved asset utilization	Economic modeling and marketplace analysis	Highlighted the importance of trust mechanisms and transparent pricing in rental marketplace success
IJERT Research Team (2022)	Peer-to-peer online rental platform development	Designed and implemented a P2P rental platform with basic listing and search functionality	React.js, Node.js, MongoDB	Demonstrated feasibility of web-based rental platforms but was limited to single category without verification systems
Hirao, Morifumi (2023).	Product categories suitable for peer-to-peer rental markets	Identified that items with high purchase prices and intermittent usage are most suitable for rental marketplaces	Market research and category analysis	Supported the multi-category approach adopted in the proposed system covering cameras, vehicles, electronics, and more
Codica Development Team (2025)	System capturing user emotions via camera, suggesting music based on emotions	Provided detailed insights in architecture, design patterns, flows, and deployment strategies	Web frameworks and cloud services	Informed tech selection decisions toward cost-effective solutions using frameworks and free-tier cloud services

4. SYSTEM DESIGN

4.1 INTRODUCTION TO MODULES

The One-Stop Digital Rental Marketplace Platform is structured into multiple modules, where each module is designed to perform a specific task in the overall system workflow. This modular design ensures better organization, easy implementation, and improved system performance.

Each module operates independently but is interconnected with other modules to achieve the final objective of providing a comprehensive rental discovery and connection platform for users and item owners. The division of the system into modules also helps in debugging, testing, and future enhancements.

Overview of Modules

The system is composed of the following major modules:

1. Authentication Module

This module is responsible for managing user registration, login, session management, and OAuth integration. It supports both email and password registration and Google OAuth for one-click sign-in. The module automatically creates user profiles upon registration through database triggers and maintains authentication state across page navigations using middleware-based token refresh.

2. Listing Management Module

This module handles the complete lifecycle of rental listings from creation through management to deletion. It provides a guided seven-step creation form that walks owners through category selection, photo upload, item details, pricing, location, contact information, and final review before publishing. Owners can also edit, toggle status, and delete their listings through the dashboard.

3. Search and Discovery Module

This module enables users to find relevant rental items through multiple discovery pathways. It includes a horizontally scrollable category bar with sixteen category icons, full-text search across listing titles and descriptions, advanced filtering by city, price range, and condition, and multiple sorting options including newest, price ascending, price descending, and top rated.

4. Review and Rating Module

This module facilitates user feedback through star ratings and written text reviews. It automatically calculates and updates average ratings for both individual listings and owner profiles whenever reviews are created, updated, or deleted. The module ensures that only authenticated users can submit reviews and prevents owners from reviewing their own listings.

5. Wishlist Module

This module allows authenticated users to save interesting listings to their personal wishlist for future reference. It uses optimistic UI updates for instant visual feedback when toggling the heart icon and synchronizes the state with the database. Users can view all saved items on a dedicated wishlist page..

6. Contact and Communication Module

This module facilitates direct communication between renters and item owners. It provides a contact modal that displays the owner's phone number with a call button, WhatsApp number with a pre-filled message link, and email address with a compose button. The module tracks contact inquiries by incrementing the inquiry count on listings.

7. Owner Dashboard Module

This module provides analytics and management tools for listing owners. It displays key metrics including total views, total inquiries, average rating, and listing performance comparisons. Owners can manage their listings by toggling status between active, rented, and inactive states, and can delete listings with confirmation dialogs.

8. Navigation and Layout Module

This module provides consistent navigation, category browsing, and page layout across the entire application. It includes the sticky navbar with logo, search bar, and user menu dropdown, the horizontal category bar, the multi-column footer, and responsive layout wrappers that adapt to different screen sizes.

9. Professional Pages Module

This module delivers informational content for platform credibility and user onboarding. It includes the About page with mission statement and statistics, How It Works page with step-by-step guides, Contact page with support form, FAQ page with expandable accordion items, Terms and Conditions page, and Privacy Policy page.

Module Interaction

- Users authenticate through the Authentication Module
- Authenticated users create listings through the Listing Management Module
- All users discover items through the Search and Discovery Module
- Users view detailed information and contact owners through the Contact Module
- Users save favorites through the Wishlist Module
- Users provide feedback through the Review and Rating Module
- Owners manage and analyze performance through the Dashboard Module
- Results are displayed consistently through the Navigation and Layout Module

Modules Description

1. **Authentication Module**

Manages user registration, login, Google OAuth, and session management

2. **Listing Management Module**

Handles seven-step listing creation, editing, status management, and deletion

3. **Search and Discovery Module**

Provides category browsing, text search, advanced filtering, and sorting.

4. **Review and Rating Module**

Enables star ratings and written reviews with automatic average calculation.

5. **Wishlist Module**

Allows users to save and manage favorite listings with optimistic UI updates

6. **Contact and Communication Module**

Facilitates direct owner contact via phone, WhatsApp, and email

7. **Owner Dashboard Module**

Provides listing analytics, status management, and performance metrics

8. **Navigation and Layout Module**

Delivers consistent navbar, category bar, footer, and responsive layouts

9. **Professional Pages Module**

Presents About, How It Works, FAQ, Contact, Terms, and Privacy pages.

4.2 MODULE IMPLEMENTATION

The implementation of modules in the One-Stop Digital Rental Marketplace Platform focuses on integrating frontend web development, backend-as-a-service, cloud database management, and responsive user interface design to achieve a production-grade application. Each module is developed using appropriate tools and technologies to ensure scalability, security, and smooth interaction between components.

Step-by-Step Process Flow:

1. Authentication Module Implementation

- Implemented using Supabase Auth with `@supabase/ssr` package for Next.js integration
- Supports email and password registration with automatic profile creation via database trigger
- Google OAuth login configured through Google Cloud Console credentials and Supabase provider settings
- AuthProvider React context wraps the entire application and manages user state, profile fetching, and auth methods
- useAuth custom hook provides convenient access to authentication state & functions throughout app
- Auth callback route handler exchanges OAuth authorization codes for sessions and redirects users
- Next.js middleware refreshes authentication tokens on every request to maintain persistent login state
- ProtectedRoute wrapper component checks authentication before rendering protected pages

2. Listing Management Module Implementation

Uses Zustand state store to manage form data across seven steps of listing creation

Step 1 fetches categories from Supabase and displays them as selectable icon cards in a responsive grid

Step 2 uses react-dropzone for drag-and-drop photo upload to Supabase Storage bucket with preview thumbnails

Step 3 provides text inputs for title, description, brand, model, and condition dropdown with character counters

Step 4 offers price inputs for hourly, daily, weekly, and monthly rates with optional security deposit

Step 5 captures city, area, state, and pincode with autocomplete suggestions for major Indian cities

Step 6 collects phone number, WhatsApp number, email, preferred contact method, and rental terms

Step 7 displays a comprehensive review of all entered data with edit buttons for each section

On publish, listing data is inserted into the Supabase listings table and user is redirected to the new listing page

My Listings dashboard queries listings by owner ID and provides status tabs, action buttons, and performance metrics

3. Search and Discovery Module Implementation

- Category bar component fetches all sixteen categories from Supabase and renders them with corresponding Lucide React icons
- Icon mapping function converts database icon name strings to actual Lucide React components
- Scroll arrows appear dynamically based on scroll position for horizontal navigation
- URL search parameters track selected category, search query, city, price range, and sort order
- Full-text search utilizes PostgreSQL tsvector column for efficient text matching across multiple fields
- Filter panel provides category checkboxes, price range inputs, city selection, condition filters, and sort options
- Infinite scroll pagination uses Supabase range queries to load additional listings as user scrolls
- Results count and active filter chips are displayed above the listing grid

4. Review and Rating Module Implementation

- Review modal component provides star rating selector and text comment input with validation
- Reviews are inserted into the Supabase reviews table with listing ID, reviewer ID, owner ID, rating.
- Database trigger function automatically recalculates average rating and total review count for the listing
- Owner profile aggregate ratings are also updated through the same trigger function
- Reviews are displayed in a two-column grid on listing detail pages with reviewer avatar, name, date, and rating
- Duplicate reviews from the same user on the same listing are prevented by unique constraint

5. Wishlist Module Implementation

- Heart icon on listing cards toggles wishlist state using Supabase insert and delete operations
- Optimistic UI update immediately changes heart color before database confirmation
- If user is not authenticated, clicking the heart opens the authentication modal instead
- `stopPropagation` prevents heart click from navigating to the listing detail page
- Wishlist page queries the wishlists table joined with listings, profiles, and categories for data
- Removing items from wishlist provides toast notification and updates the displayed list

6. Contact and Communication Module Implementation

- Contact modal displays owner avatar, name, and response time information
- Phone number is shown with a clickable tel: link for direct calling
- WhatsApp link is generated in the format wa.me with URL-encoded pre-filled message
- Email contact uses mailto: link with the listing title in the subject line
- Inquiry count is incremented through a Supabase database function when the contact modal opens
- Safety tips section advises users on verification, receipts, and meeting arrangements

7. Owner Dashboard Module Implementation

- Dashboard page queries all listings where owner ID matches the current authenticated user
- Status filter tabs display counts for All, Active, Rented, and Inactive listings
- Each listing card shows thumbnail image, title, status badge, view count, inquiry count, and rating
- Action buttons provide Edit, Toggle Status, and Delete functionality with confirmation dialogs
- Delete operation removes the listing from the database and associated images from Supabase
- Create New Listing button provides quick access to the listing creation form

8. Navigation and Layout Module Implementation

- Navbar component includes ORMA logo in coral color, search pill bar, List Your Item link.
- User menu dropdown shows different options for authenticated and unauthenticated users
- Category bar uses useRef for scroll container and useState for scroll arrow visibility
- Footer component provides four-column layout with navigation links to all platform pages
- ClientLayout component manages auth modal state and category selection as a client-side wrapper
- Root layout provides AuthProvider, Toaster for notifications, and proper metadata configuration

9. Professional Pages Module Implementation

- About page includes hero section, mission statement, differentiator cards, statistics section.
- How It Works page provides step-by-step guides for both renters and owners with safety tips
- Contact page features a form with name, email, subject dropdown, message textarea.
- FAQ page uses useState for accordion expand and collapse with categorized questions and answers
- Terms and Privacy pages present legal content in clean prose layout with table of contents navigation

Integration of Modules

- All modules are integrated through the Next.js App Router file-based routing system
- Data flows between modules through Supabase client queries and React context providers
- Authentication state is shared across all modules through the AuthProvider context
- URL search parameters maintain state across page navigations for search and filter operations
- Toast notifications provide consistent user feedback across all module interactions
- Error boundaries and loading states ensure graceful handling of failures throughout the application

Conclusion

The implementation of modules ensures that the system operates efficiently as a production-grade web application. By combining Next.js for server-side rendering, Supabase for backend services, Tailwind CSS for responsive styling, and Zustand for state management, the system delivers a comprehensive and user-friendly rental marketplace experience. The modular implementation also allows easy upgrades and integration of advanced features in the future.

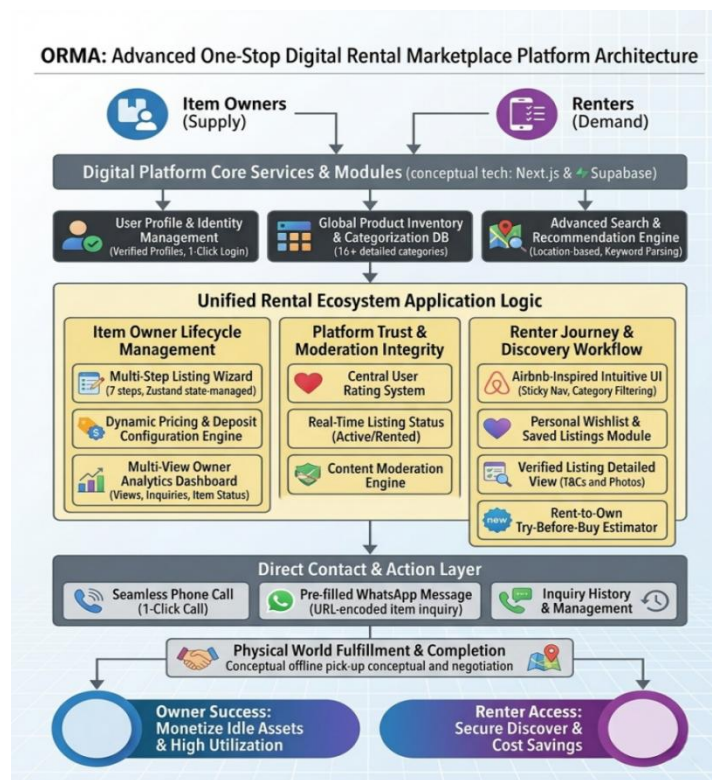
5. SYSTEM DESIGN

5.1 Introduction to System Design

System design plays a crucial role in the development of the One-Stop Digital Rental Marketplace Platform. It involves designing the architecture, process flow, data handling mechanisms, and user interactions that enable efficient listing management, rental discovery, profile handling, communication, and dashboard analytics. The system must be designed to handle real-time listing access, secure authentication, image storage, search operations, and responsive user interaction efficiently.

The design process includes frontend-backend integration, database schema planning, page navigation structure, component organization, and cloud service connectivity. Additionally, scalability, reliability, usability, and maintainability are key factors that influence the system design. A well-structured system ensures smooth operation, seamless communication between components, and effective user interaction through browser-based interfaces.

The ORMA platform is designed as a modern web application that supports direct interaction between renters and owners without including payment processing in the current version. The design emphasizes product visibility, user convenience, structured discovery, and trust-building features. By integrating category navigation, listing management, search and filters, owner contact options, and review systems, the design creates a complete and practical rental discovery ecosystem.



5.2 Process Flow of Project

The system follows a structured end-to-end process flow that involves multiple interconnected modules working together. The flow of the ORMA rental marketplace platform can be explained through the following steps:

Step 1: User Access and Initialization

User opens the platform in a browser

Home page, categories, and active listings are loaded

Authentication state is checked

Step 2: User Authentication

If the user wants protected features, they can sign up or log in

User session is created and profile data is retrieved

Authenticated users gain access to listing creation, wishlist, dashboard, and messaging

Step 3: Listing Discovery

User browses categories or uses the search bar

Filters such as city, category, and price are applied

Matching listings are displayed in card format

Step 4: Listing Details Viewing

User opens a product details page

Platform displays images, pricing, description, owner profile, terms, and reviews

Similar items and owner information are also shown

Step 5: Contact and Interaction

User chooses to call, WhatsApp, or message the owner

Inquiry count or message activity is updated

User may also save the item to wishlist or review it later

Step 6: Owner Listing Management

Authenticated owner opens dashboard

Owner creates new listings or edits existing ones

Owner monitors status, inquiries, views, and ratings

Step 7: Review and Trust Building

Users submit ratings and reviews

Platform updates average ratings and trust-related fields

Public profiles reflect review-based credibility

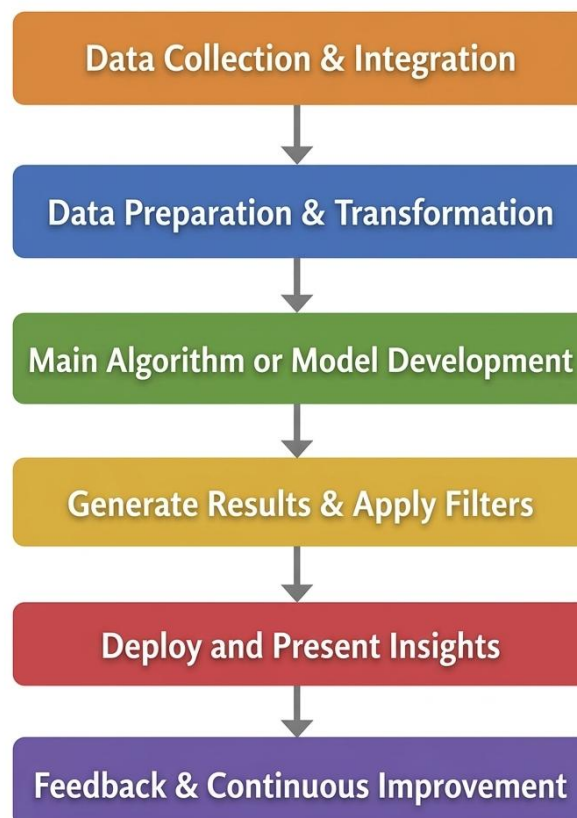
Step 8: Output and Continuity

Platform continuously supports browsing, search, listing updates, and communication

User can repeat the same cycle for other products and categories

This process flow ensures that rental listing, discovery, & owner-renter interaction in organized manner.

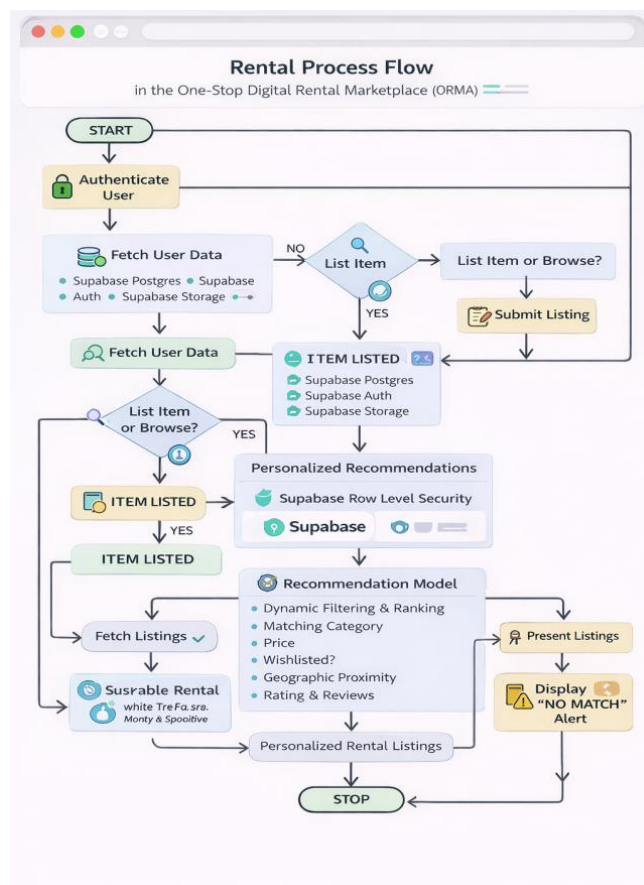
Project Process Flow



5.3 Flowchart Diagram

A flowchart provides a structured representation of the logical flow of operations within the ORMA platform. The flowchart of this system follows these steps:

- Start
- Open Platform
- Check User Authentication State
- Display Home Page with Categories and Listings
- User Browses or Searches for a Product
- Apply Filters and Fetch Matching Listings
- Select Listing
- Display Detailed Product Information
- Decision: User Logged In?
 - If No → Show Auth Prompt for protected actions
 - If Yes → Allow Wishlist / Messaging / Review
- User Contacts Owner or Saves Listing
- Owner Manages Listing through Dashboard
- Update Database Records (views, inquiries, reviews, status)
- Display Updated Results
- End / Continue Browsing



5.4 Use Case Diagram

The Use Case Diagram represents the interaction between the actors and the ORMA platform.

The main actors involved in the system include:

Actors

- Renter / User
- Owner / Vendor
- Authentication Service
- Database / Storage Service

Main Use Cases for User

- Register / log in
- Register account
- Log in
- Browse categories
- Search listings
- Filter listings
- View product details
- Save item to wishlist
- Contact owner
- Send message
- Submit review
- View public profiles

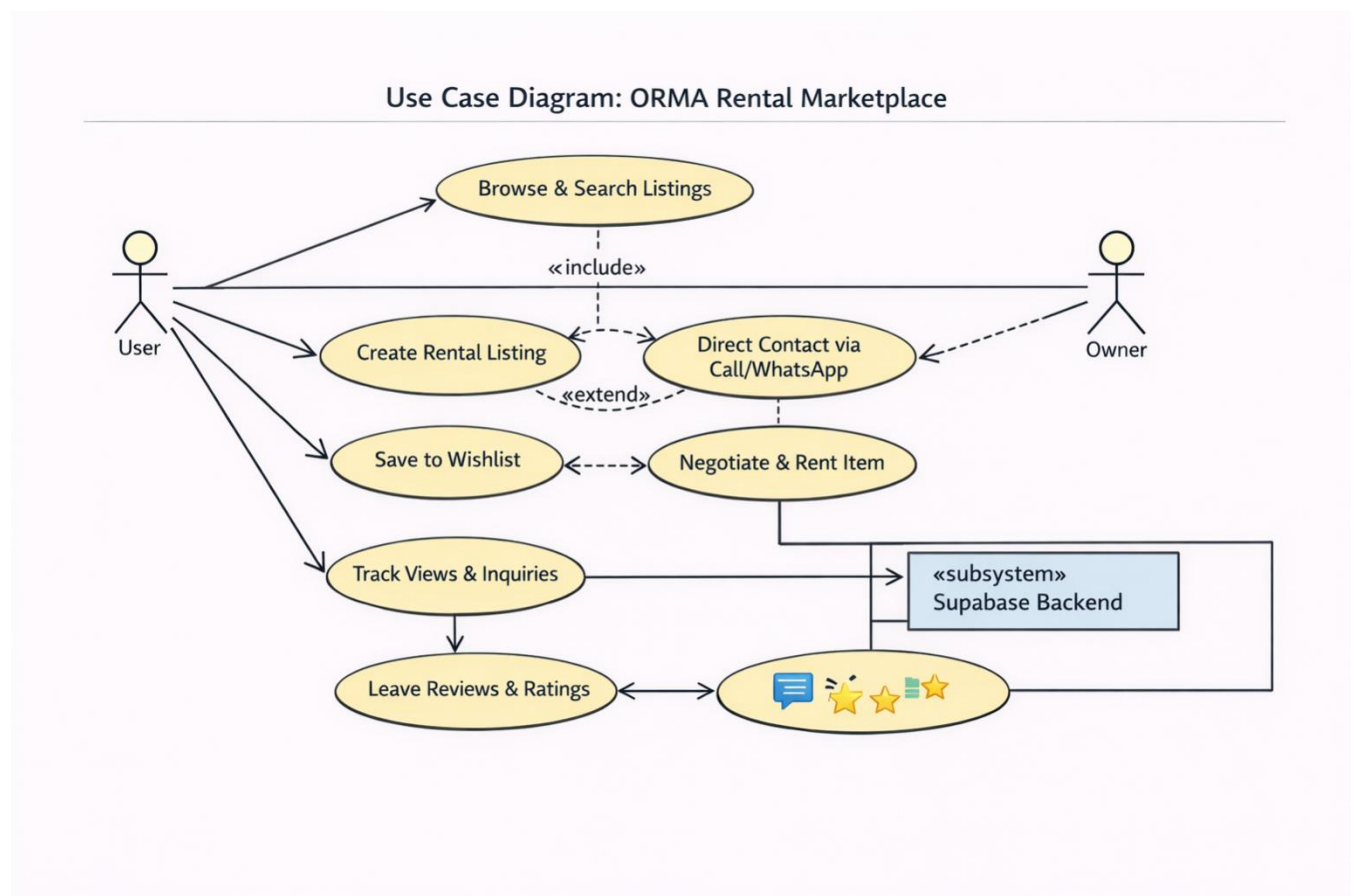
Main Use Cases for Owner

- Update Database Records (views, inquiries, reviews, status)
- Display Updated Results
- End / Continue Browsing
- Register / log in
- Create listing
- Upload images
- Edit listing
- Delete listing
- Change listing status
- View dashboard
- Check inquiries
- View analytics
- Manage profile

Main Use Cases for System

- Authenticate users
- Store listings and images
- Retrieve search results
- Maintain reviews and ratings
- Manage wishlists
- Maintain conversations and inquiries

The use case diagram shows that renters and owners share some common functions such as authentication and profile access, while owners additionally manage listings and dashboards.



5.5 Sequence Diagram

A Sequence Diagram depicts the sequential interaction between components in the ORMA

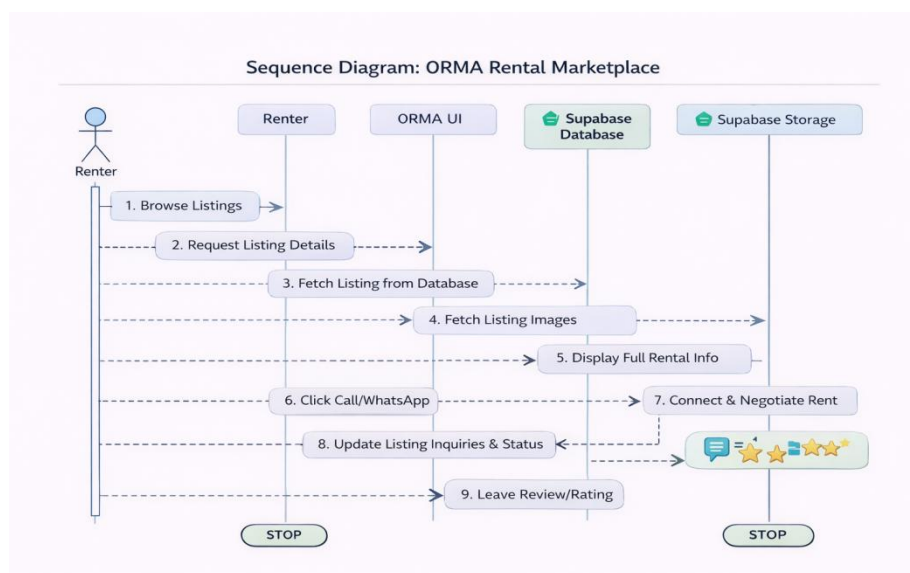
Typical Sequence: Listing Discovery and Contact

- User opens home page
- UI requests active listings from database
- Database returns listings with category and owner details
- User selects a listing
- UI requests detailed listing data
- Database returns listing, owner profile, and reviews
- User clicks contact owner
- System checks authentication state
- If authenticated, contact options or message module opens
- Inquiry / message record is stored
- Owner receives inquiry in dashboard / messages
- User continues browsing or exits

Typical Sequence: Listing Creation

- Owner logs in
- Owner opens “List Your Item” page
- Owner fills multi-step listing form
- Images are uploaded to cloud storage
- Listing data is submitted
- Database stores listing record
- Confirmation is shown
- Listing becomes visible on marketplace feed

The sequence diagram helps show how the frontend, backend, storage, and user actions interact in time order.



5.6 Activity Diagram

Activity Flow: Rental Discovery

- Start
- User visits platform
- User browses categories or enters search term
- System loads matching listings
- User selects listing
- System shows listing details
- User decides whether to save, contact, or continue browsing
- If save → add to wishlist
- If contact → open phone / WhatsApp / message flow
- If continue → return to listings
- End

Activity Flow: Owner Listing Management

- Start
- Owner logs in
- Owner opens dashboard
- Owner creates or edits listing
- System validates form data
- Images are uploaded
- Listing is saved in database
- Listing status is updated
- Dashboard refreshes
- End

This activity diagram makes the operational steps of browsing and listing management easy to visualize.

5.7 Activity Diagram

A Class Diagram represents the static structure of the system by showing the main entities, their attributes, and their relationships. In ORMA, the core classes are derived from the marketplace data model.

Main Classes in the System

1. Profile

Attributes:

- id

- full_name
- email
- avatar_url
- phone
- city
- state
- bio
- is_verified
- created_at

Methods/Category:

- id
- name
- slug
- icon_name
- display_order
- register()
- updateProfile()
- viewPublicProfile()
- fetchCategories()
- filterListingsByCategory()

Listing Attributes::

- id
- owner_id
- category_id
- title
- description
- condition
- images
- price_per_day
- city
- area
- contact_phone
- terms_and_conditions
- status
- is_available
- average_rating

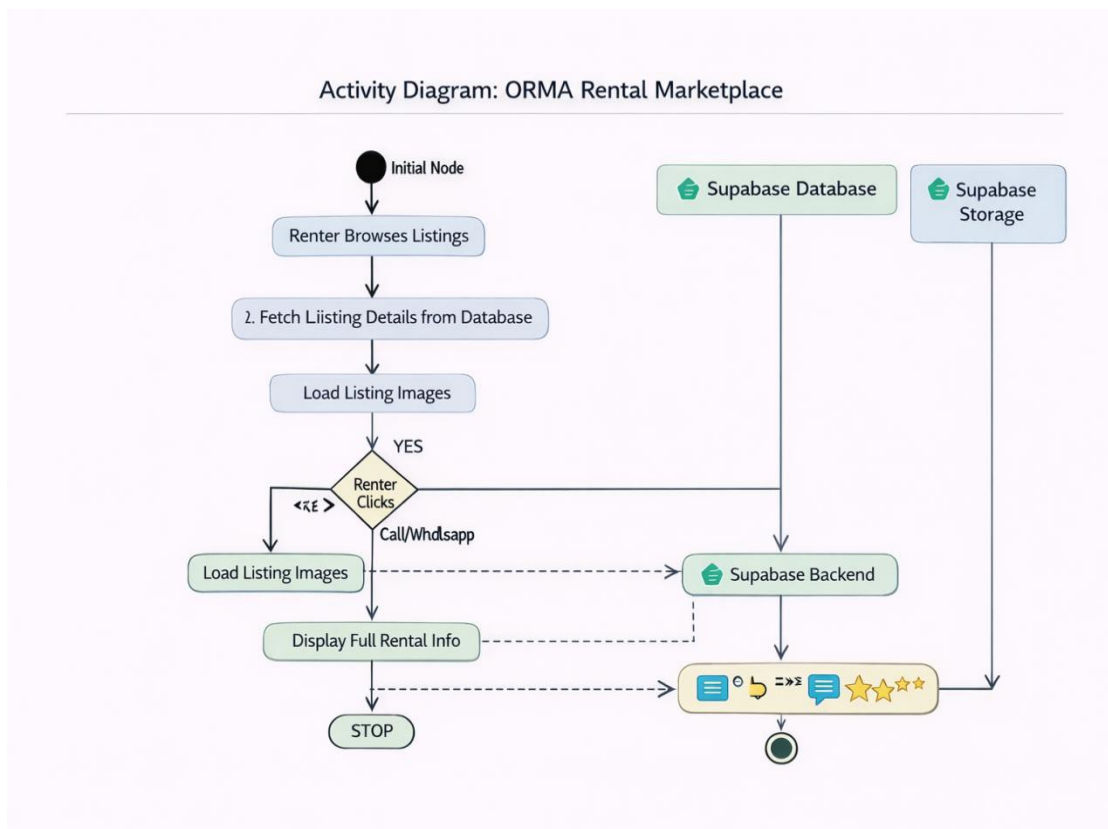
Dashboard:

- average_rating
- owner_id
- views_count
- inquiries_count
- average_rating
- listing_count
- fetchAnalytics()
- showOwnerStats()

Relationships:

- One Profile can own many Listings
- One Category can contain many Listings
- One Listing can have many Reviews
- One User can have many Wishlist items
- One Listing can have many Inquiries
- One Owner Dashboard summarizes data from many listing

The class diagram helps define the object-level model behind the system and supports easier implementation.



5.8 State Diagram

The Real-Time Emotion Detection System is the most important component of the project. It is responsible for analyzing the user's facial expressions and identifying their emotional state. This system uses computer vision and deep learning techniques to recognize emotions quickly and accurately.

Example:

Listing State Diagram

A rental listing in ORMA can move through the following states:

Draft

Listing form is being filled but not yet published

Published / Active

Listing is visible on the platform and available to renters

Unavailable / Rented

Listing is temporarily not available for rental

Inactive

Owner deactivates the listing manually

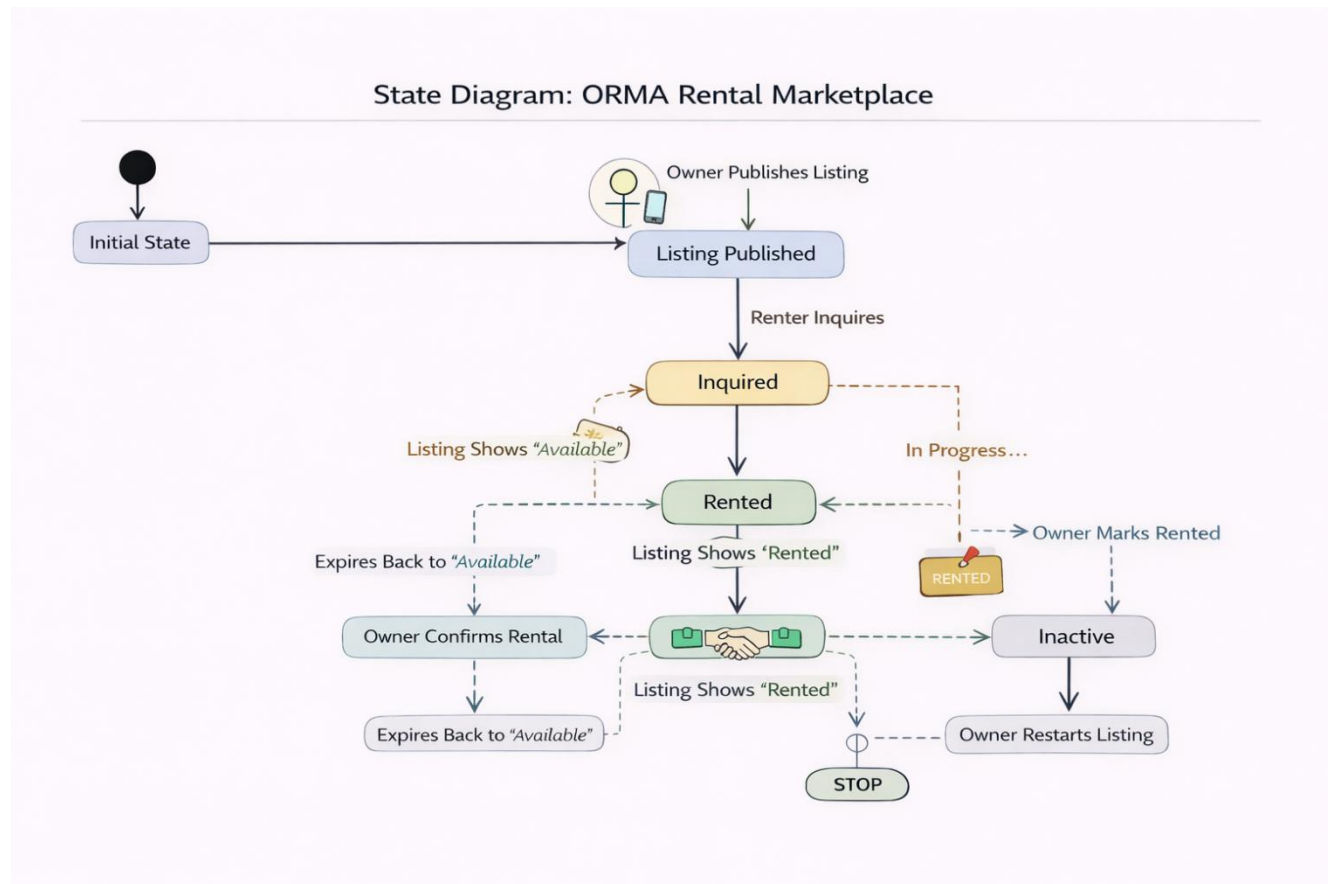
Deleted

Listing is removed permanently from the platform

State Transitions:

- Draft → Active (after publishing)
- Active → Unavailable (when rented or disabled temporarily)
- Active → Inactive (manual deactivation)
- Inactive → Active (reactivation)
- Any state → Deleted (owner deletion)
- Example: User Session State Diagram
- Logged Out
- Login Requested
- Authenticated
- Session Expired / Logged Out

This diagram helps represent how important entities such as listings and user sessions behave over time.



5.9 Component Diagram

The Component Diagram represents the high-level architecture of the platform and shows how different software components are connected.

Main Components

1. Client Application

Built with Next.js and React

Handles user interaction and UI rendering

2. Authentication Component

Supabase Auth

Handles login, signup, session management

3. Listing Service Component

Manages listing creation, retrieval, update, and deletion

4. Search & Filter Component

Handles search queries and filter application

5. Review & Wishlist Component

Manages saved items, ratings, and reviews

6. Messaging Component

Handles user-owner communication and inquiry records

7. Dashboard Component

Displays owner analytics and listing performance

8. Database Component

PostgreSQL database via Supabase

Stores all relational data

9. Storage Component

Supabase Storage

Stores listing images

10. Deployment Component

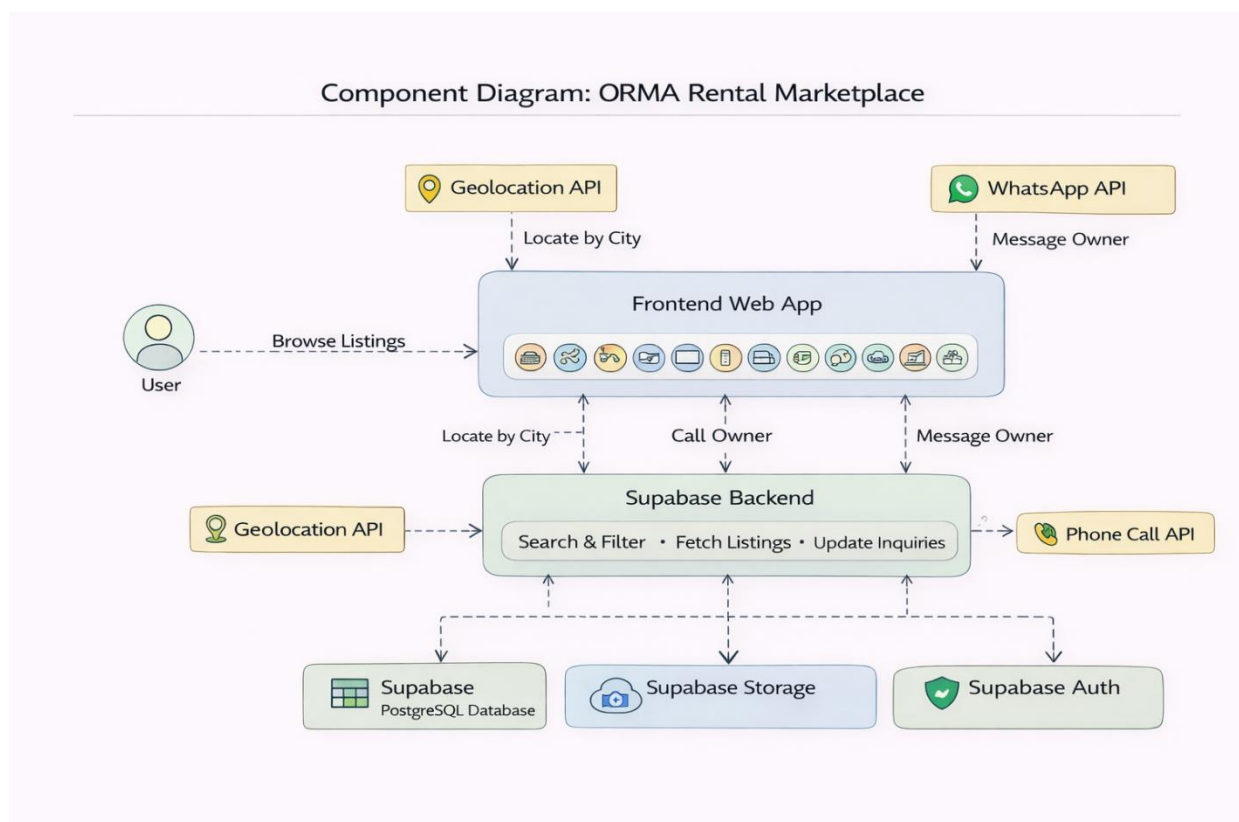
Vercel hosting environment

Serves production web application

Component Interaction

- Client Application communicates with authentication, listing, review, wishlist, search, and messaging components
- These components interact with the database
- Listing service also interacts with storage for image upload
- Dashboard component retrieves aggregate listing data
- Deployment component hosts the final application

The component diagram provides a clear overview of how different parts of the ORMA platform are connected and work together. It ensures modular design, scalability, and efficient communication between components.



6.SYSTEM IMPLEMENTATION

6.1 Introduction to System Implementation

System implementation is the phase in which the designed system is developed and converted into a working application. In this stage, all the modules that were planned during the system design phase are created and integrated to build the final system. The main goal of implementation is to ensure that the system functions correctly and performs the tasks it was designed to do.

In the One-Stop Digital Rental Marketplace Platform project, system implementation involves developing the modules responsible for user authentication, listing management, search and discovery, review systems, wishlist functionality, contact capabilities, and the owner dashboard. The implementation process begins with selecting appropriate programming languages, frameworks, and cloud services. These tools are used to write the code that controls how the system handles user registration, listing creation, image uploads, database queries, and responsive interface rendering.

The implementation also includes building the user interface that allows users to interact with the platform. The interface provides options to browse listings, search for items, view detailed listing pages, contact owners, create new listings, and manage personal profiles. All these components are connected together so that the system works smoothly across desktop and mobile devices.

Another important part of system implementation is integrating the backend-as-a-service platform with the frontend application. Supabase provides the PostgreSQL database, authentication services, and file storage that power the platform, while Next.js handles server-side rendering and client-side interactivity..

6.2 Selection of Software

Choosing the appropriate software tools is an important step in system development. The selected tools should support the required functionalities and make development easier and more efficient. In this project, several software tools and platforms are used to implement the system.

The primary development framework used is Next.js, which is a React-based web application framework widely used for building modern, production-grade web applications. Next.js provides server-side rendering, file-based routing, API route handling, and optimized performance out of the box. Its App Router architecture enables efficient code splitting and streaming for faster page loads.

For backend services including database, authentication, and file storage, Supabase is used. Supabase is an open-source Backend-as-a-Service platform that provides a managed PostgreSQL database, built-in authentication with multiple providers, object storage for files, and real-time data subscriptions. Using Supabase eliminates the need to build and maintain a custom backend server, significantly reducing development time and complexity.

For deployment and hosting, Vercel is used. Vercel is a cloud platform specifically designed for Next.js applications that provides automatic deployments from GitHub, global CDN distribution, serverless function support, and custom domain configuration with automatic SSL certificates..

6.3 Programming Languages and Frameworks

Programming languages and frameworks form the foundation of any software system. In this project, different technologies are used to handle various tasks such as user interface rendering, data management, state handling, and responsive styling.

The primary programming language used in this project is TypeScript. TypeScript is a strongly-typed superset of JavaScript that provides compile-time type checking, enhanced code intelligence, and improved developer productivity. TypeScript catches errors during development rather than at runtime, reducing bugs and improving code reliability. All components, utilities, hooks, and type definitions in the project are written in TypeScript.

The frontend framework used is Next.js with the App Router architecture. Next.js provides server components for static content and metadata rendering, and client components for interactive elements that require state management, event handlers, or browser APIs. The file-based routing system automatically creates URL routes.

For styling the user interface, Tailwind CSS is used. Tailwind CSS is a utility-first CSS framework that enables rapid development through composable utility classes applied directly in the JSX markup. The Airbnb-inspired design system including the coral primary color, card shadows, border radii, and responsive breakpoints is implemented entirely through Tailwind CSS configuration and utility classes.

For client-side state management, Zustand is used specifically for the multi-step listing creation form. Zustand provides a lightweight and simple API for managing complex form state across seven form steps without the boilerplate required by larger state management libraries.

6.4 Backend-as-a-Service

The backend infrastructure of the One-Stop Digital Rental Marketplace Platform is powered entirely by Supabase, which serves as a comprehensive Backend-as-a-Service platform. Supabase provides several critical services that enable the platform to function without a custom backend server.

The primary service provided by Supabase is the managed PostgreSQL database. PostgreSQL is a powerful open-source relational database that stores all platform data including user profiles, rental categories, listings, reviews, wishlists, inquiries, conversations, and messages. The database schema includes carefully designed tables with proper foreign key relationships, indexes for query optimization, full-text search vectors for efficient search functionality, and Row Level Security policies that control data access at the database level.

Supabase Auth provides the authentication service for the platform. It supports email and password registration, Google OAuth integration, session management with JWT tokens, and automatic token refresh. When a new user registers, a database trigger function automatically creates a corresponding profile record in the profiles table, populating it with the user's name, email, and avatar URL from the authentication metadata.

Supabase Storage provides object storage for listing images and user avatars. Images uploaded during the listing creation process are stored in a public storage bucket with appropriate access policies. Authenticated users can upload and manage their own

6.5 Database and Storage Solutions

The database and storage system is responsible for storing and managing all information required for the rental marketplace platform. In this project, the storage components maintain details about user profiles, rental categories, listings, reviews, wishlists, and inquiries. Proper database design ensures that the system can quickly retrieve and display rental listings, search results, and user-specific data.

In the One-Stop Digital Rental Marketplace Platform, the database consists of six core tables. The profiles table stores user information including name, email, avatar, phone, city, bio, and verification status. The categories table contains sixteen predefined rental categories with names, slugs, icon mappings, and display order. The listings table is the main data table storing all rental item information including title, description, images, pricing, location, contact details, terms, status, and metrics.

The reviews table stores user feedback with star ratings, comments, and references to both the listing and the reviewer. The wishlists table tracks which listings each user has saved to their favorites. The inquiries table records contact requests between renters and owners for tracking and analytics purposes.

The storage system uses Supabase Storage with a public bucket named listing-images. When owners upload photos during listing creation, the images are stored in a folder structure organized by user ID and timestamp. The system generates public URLs for uploaded images that are stored in the listings table as a text array, enabling efficient image display across the platform.

Database functions and triggers automate several operations. The `handle_new_user` function creates profile records automatically upon registration. The `update_listing_rating` function recalculates average ratings whenever reviews are added, updated, or deleted. The `increment_views` and `increment_inquiries` functions track listing engagement metrics.

6.6 Authentication and Security System

The authentication and security system is a critical component of the platform that ensures only authorized users can perform protected actions such as creating listings, submitting reviews, saving to wishlists, and contacting owners. The system uses Supabase Auth combined with Next.js middleware for session management.

The authentication flow begins when a user clicks the Log In button in the navbar, which opens a modal popup with tabbed Login and Sign Up forms. For email registration, the user provides their full name, email, password, and password confirmation. The system validates inputs, creates the authentication record through Supabase Auth, and the database trigger automatically generates the user profile.

For Google OAuth login, clicking the Continue with Google button redirects the user to Google's authentication page. After successful authentication, Google redirects back to the application's auth callback route, which exchanges the authorization code for a session token and redirects the user to the home page.

Row Level Security policies on all database tables ensure that users can only read public data and modify their own records. For example, any user can view active listings and published reviews, but only the listing owner can update or delete their own listings, and only the review author can modify their own reviews.

6.7 User Interface Development

User Interface development is an important part of any software system because it allows users to interact with the application easily. A good user interface should be simple, clear, and easy to understand so that users can operate the system without confusion. In the One-Stop Digital Rental Marketplace Platform, the user interface is designed to provide a smooth, modern, and responsive experience inspired by Airbnb's highly successful design patterns.

The main goal of the user interface is to display rental listings in an attractive card-based grid, provide intuitive navigation through categories, enable efficient search and filtering, and present detailed listing information with clear contact options. The interface acts as a bridge between the user and the backend system that manages listings, authentication.

The UI is built using Next.js React components styled with Tailwind CSS utility classes. The navbar component includes the ORMA logo in coral color, a search pill bar in the center, and a user menu dropdown on the right side. The category bar displays sixteen rental category icons in a horizontally scrollable container with smooth scroll arrows. The listing card component shows item images in a carousel with wishlist heart icons, location, title, category, price, and rating information.

Framer Motion is used to add smooth animations throughout the application including staggered card loading, page transitions, and interactive micro-interactions on buttons and icons. react-hot-toast provides beautiful notification messages for user actions such as successful login, listing creation, wishlist updates, and error handling.

6.8 Selection of Software

The following code snippets demonstrate key implementation patterns used in the One-Stop Digital Rental Marketplace Platform.

TypeScript

```
// Supabase Client Configuration (src/lib/supabase/client.ts)
import { createBrowserClient } from '@supabase/ssr'
export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  )
}

// Fetching Listings with Owner and Category Data
const supabase = createClient()
const { data: listings, error } = await supabase
  .from('listings')
  .select(`
    *,
    owner:profiles(*),
    category:categories(*)
  `)
  .eq('status', 'active')
  .eq('is_available', true)
```

```

.order('created_at', { ascending: false })
.limit(40)

// WhatsApp Contact Link Generation
export function getWhatsAppLink(
  phone: string,
  itemTitle: string
): string {
  const cleanPhone = phone.replace(/^\D/g, "")
  const message = encodeURIComponent(
    `Hi, I'm interested in renting your
    "${itemTitle}" listed on ORMA.`
  )
  return `https://wa.me/91${cleanPhone}?text=${message}`
}

// Price Formatting Utility for Indian Rupees
export function formatPrice(amount: number): string {
  return new Intl.NumberFormat('en-IN', {
    style: 'currency',
    currency: 'INR',
    maximumFractionDigits: 0,
  }).format(amount)
}

// Database Trigger: Auto-create profile on signup
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO public.profiles
    (id, full_name, email, avatar_url)
  VALUES (
    NEW.id,
    COALESCE(
      NEW.raw_user_meta_data->>'full_name', ""
    ),
    COALESCE(NEW.email, ""),
    COALESCE(
      NEW.raw_user_meta_data->>'avatar_url', ""
    )
  )

```

```

);
RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW
  EXECUTE FUNCTION public.handle_new_user();

```

```

// Category Icon Mapping for Lucide React
const iconMap: Record<string, LucideIcon> = {
  'Car': Car,
  'Bike': Bike,
  'Camera': Camera,
  'Laptop': Laptop,
  'Smartphone': Smartphone,
  'Armchair': Armchair,
  'WashingMachine': WashingMachine,
  'Gamepad2': Gamepad2,
  'Wrench': Wrench,
  'Dumbbell': Dumbbell,
  'Music': Music,
  'PartyPopper': PartyPopper,
  'BookOpen': BookOpen,
  'Plane': Plane,
  'Shirt': Shirt,
  'Package': Package,
}

```

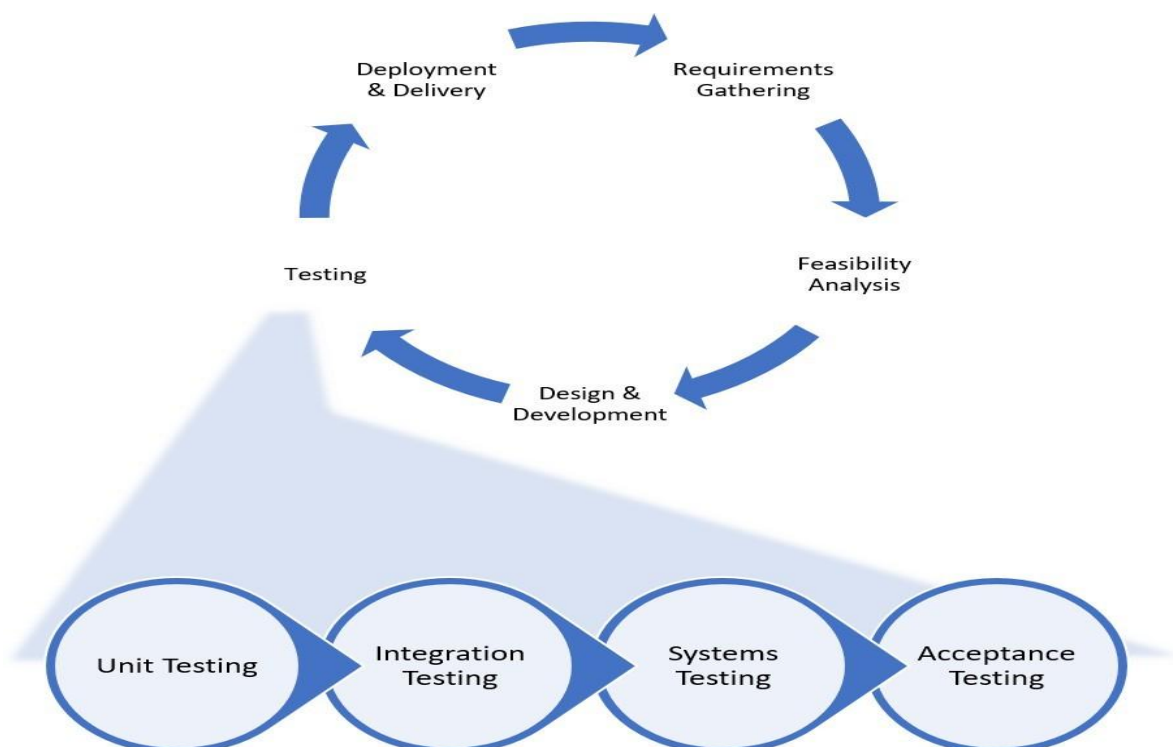
7 . TESTING

7.1 Introduction

Software testing is an important phase in the software development process. It is used to verify that the system works correctly and meets the requirements specified during the design and development stages. Testing helps identify errors, bugs, and problems in the system before it is used by real users. The main objective of testing is to ensure that the system performs its intended functions accurately, reliably, and efficiently.

In the One-Stop Digital Rental Marketplace Platform (ORMA), testing plays a significant role in verifying that each module of the system works properly. The system includes several components such as user authentication, listing management, search and filtering, contact owner functionality, review system, and wishlist management. Each of these components must be tested carefully to ensure that they function correctly and interact with each other without errors.

Testing also helps confirm that the system can correctly authenticate users, display rental listings with accurate information, enable category-based filtering, and facilitate direct contact between renters and item owners. For example, when a user clicks on a category such as "Cameras," the system should display only camera-related rental listings. Similarly, when a user clicks "Contact Owner" on a listing detail page, the system should display the correct phone number and WhatsApp link for that specific owner. Testing ensures that the system responds correctly to all user interactions across different pages and features.



7.2 Test Cases

Test cases are used to verify whether the system behaves as expected under different conditions. A test case is a set of conditions or inputs that are used to check the functionality of a specific part of the system. Each test case includes the input data, the expected output, and the actual output produced by the system.

Test Case ID	Module	Test Description	Input	Expected Result	Status
TC01	Login	Verify login with valid credentials	Correct username & password	User logged into system	Pass
TC02	Login	Verify login with invalid credentials	Wrong username/password	Error message displayed	Pass
TC03	Listing Creation	Create new listing through multi-step form	Complete all 7 steps with valid data	Listing published and visible on home page	Pass
TC04	Searching	Search listings by keyword	Enter "Canon" in search bar	Listings with "Canon" in title displayed	Pass
TC05	Wishlist	Save listing to wishlist	Click heart icon on listing card	Heart turns red, item added to wishlist page	Pass
TC06	Review System	Submit review on listing	Enter rating and comment, click submit	Review displayed on listing detail page	Pass

7.3 Unit Testing

Unit testing is a testing method where individual components or modules of the system are tested separately. The main objective of unit testing is to verify that each module performs its intended function correctly without errors.

In the One-Stop Digital Rental Marketplace Platform, unit testing is applied to different modules such as the authentication module, listing management module, search module, and review module. Each module is tested independently before integrating them into the complete system. This approach helps identify and fix issues at the component level before they can cause problems in the integrated application

Module	Test Condition	Expected Output	Result
Contact Module	Open contact modal for listing	Owner phone and WhatsApp displayed correctly	Pass
Profile Module	Update user profile information Profile	Data updated in database, changes reflected on page	Pass
Search and Filter Module	Apply category filter to listings query	Only listings matching selected category returned	Pass
Listing Management Module	Create listing with all required fields	Listing inserted into database with correct owner ID	Pass

7.4 Integration Testing

Integration testing is performed after the completion of unit testing. The main purpose of integration testing is to verify that different modules of the system work together properly when they are combined into a single system. While unit testing focuses on testing individual modules, integration testing ensures that communication between these modules is smooth and error-free.

In the One-Stop Digital Rental Marketplace Platform, several modules need to interact with each other in order for the system to function correctly. These modules include the authentication module, listing management module, search and filter module, contact module, review module, and the user interface module. Each module performs a specific task, but the system can only function properly if these modules communicate effectively with one another.

During integration testing, developers verify whether the output of one module is correctly passed as input to another module. For example, the authentication module verifies the user's identity and establishes a session. This session information is then used by the listing management module to determine which user is creating a listing, ensuring the correct owner ID is assigned. The listing data is then used by the search module to display items on the home page. When a user clicks on a listing, the item detail module fetches the complete listing data along with the owner's profile information from the database.

Integration testing ensures that this entire process happens smoothly without errors. If any module fails to send or receive data correctly, the system will not work as expected. For example, if the authentication module does not correctly pass the user session to the listing creation module, the system may fail to assign the correct owner to a new listing. Similarly, if the review module does not correctly trigger the database function that recalculates average ratings, the listing detail page may display outdated rating information.

7.5 System Testing

System testing is performed after all modules have been successfully integrated. The purpose of system testing is to evaluate the complete system and ensure that it meets the functional and non-functional requirements specified during the design phase.

In the One-Stop Digital Rental Marketplace Platform, system testing involves testing the entire application as a single unit. The system is tested under real-world conditions to verify whether it performs all its tasks correctly. The testing process begins by opening the website and verifying that the home page loads with listing cards and category icons. The system then tests the complete user journey from browsing listings to contacting an owner.

During system testing, developers verify whether the entire flow works end-to-end. For example, a test user signs up with a new email, creates a listing with photos and pricing, publishes it, and verifies that the listing appears on the home page. Another test user then searches for the item, views the listing detail page, clicks the contact button, and verifies that the correct owner information is displayed. The test also confirms that the wishlist functionality saves items correctly and that reviews are properly submitted and displayed.

System testing also evaluates non-functional requirements such as page load speed, responsive design across different screen sizes, and proper error handling when network connections fail or invalid data is submitted. The system should handle all edge cases gracefully and provide appropriate feedback to users..

7.6 User Acceptance Testing

User Acceptance Testing (UAT) is the final stage of testing in the software development process. In this stage, the system is tested by real users to determine whether it meets their needs and expectations. The purpose of user acceptance testing is to verify that the system is user-friendly and functions correctly in real-life situations.

In the One-Stop Digital Rental Marketplace Platform, user acceptance testing involves allowing users to interact with the application and evaluate its performance. Users are asked to browse the website, search for rental items in their city, view listing details, and attempt to contact owners. The system is evaluated based on how easily users can navigate between pages, find relevant items, and complete desired actions.

Users also test the listing creation flow by signing up and listing a sample item for rent. They evaluate whether the multi-step form is intuitive, whether photo upload works smoothly, and whether the published listing appears correctly on the home page. Users then evaluate the contact functionality by verifying that phone numbers and WhatsApp links work correctly on both desktop and mobile devices.

During UAT, users provide feedback on the overall design, ease of navigation, clarity of information displayed, and responsiveness of the interface on different devices. For example, users verify that the category bar scrolls smoothly on mobile, that listing cards display properly on small screens, and that the contact modal is easy to use on touchscreen devices. This feedback is used to make final adjustments before deployment.

7.7 Performance Testing

Performance testing is conducted to evaluate how well the system performs under different conditions. The main objective of performance testing is to ensure that the system operates efficiently, responds quickly, and remains stable during operation.

In the One-Stop Digital Rental Marketplace Platform, performance testing focuses on the speed of page loading, database query response times, and image loading efficiency. Since the system serves multiple users simultaneously accessing listings across different categories and cities, it must handle concurrent requests efficiently without degradation in response time.

During performance testing, developers measure the time taken by the system to load the home page with listing cards, fetch search results with applied filters, render the listing detail page with images and reviews, and process listing creation with image uploads. The system should perform all these operations within acceptable timeframes so that users receive a smooth browsing experience.

Test ID	Test Scenario	Input	Expected Performance	Actual Result	Status
PT01	Authentication response	Click login with valid credentials	Login should complete within 2 seconds	User authenticated in 1.1 seconds	Pass
PT02	Mobile responsiveness	Open website on mobile device	All pages should render correctly	Responsive layout displayed properly	Pass
PT03	Category filter response	Click category icon	Listings should filter within 1 second	Filtered results displayed in 0.8 seconds	Pass

8 SCREENS & REPORTS

8.1 Introduction

Screens and reports are an important part of any software documentation because they provide a visual representation of how the system works. They help readers understand the functionality of the system by showing the different interfaces and outputs generated during the execution of the program. Screenshots provide evidence that the system has been successfully implemented and demonstrate how users interact with the application.

In the One-Stop Digital Rental Marketplace Platform (ORMA), the screens and reports section explains the graphical user interface and the outputs generated by the system at different stages. These screens show how the system displays rental listings, handles user authentication, presents listing details with owner contact information, manages the multi-step listing creation process, and provides search and filtering capabilities across multiple rental categories.

Screens are useful for understanding the flow of the application. They allow readers to see how the system responds to user actions and how information is displayed on the interface. For example, when a user opens the website, the system loads the home page with a horizontal category bar and a responsive grid of listing cards showing item images, prices, locations, and ratings. When the user clicks on a listing card, the system navigates to a detailed product page displaying the full image gallery, pricing breakdown, owner profile, reviews, and contact options.

Reports generated by the system may include information such as listing view counts, inquiry statistics, review aggregations, and owner performance metrics displayed on the analytics dashboard. These reports help track the system's usage and verify that the platform is functioning correctly as a rental discovery and connection marketplace.

8.2 Screenshots

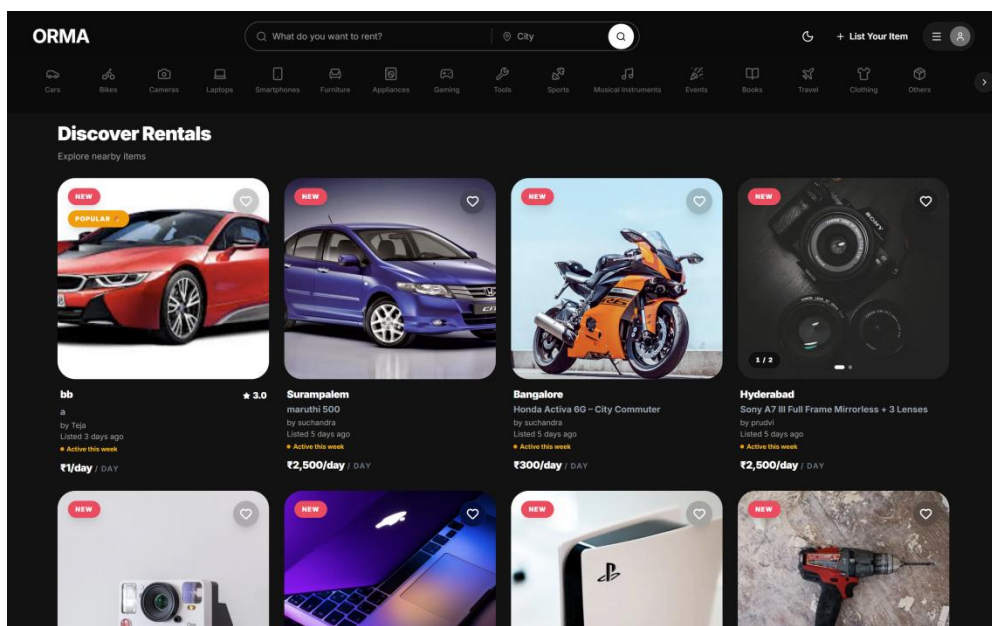
Screenshots are images captured from the system while it is running. These images help illustrate how the application works and show the different stages of the system's operation. Including screenshots in the documentation helps readers visualize the interface and understand how the system processes user input.

In the One-Stop Digital Rental Marketplace Platform project, screenshots are taken from different parts of the application to demonstrate its functionality. These screenshots include the home page with listing grid, authentication modal with login and signup tabs, listing detail page with contact owner modal, multi-step listing creation form, search results with filters, and wishlist page showing saved items.

Each screenshot represents a specific stage of the system operation. For example, one screenshot may show the home page displaying rental listings organized in a responsive card grid with category icons at the top, while another screenshot may show the listing detail page with image gallery, pricing information, and owner contact details. Additional screenshots may display the create listing form steps and the search results page with active filters.

Screenshots also help demonstrate the user interface design of the system. They show how different elements such as the navigation bar, category filter icons, listing cards with image carousels, heart-shaped wishlist buttons, and contact modals are arranged on the screen. This helps readers understand how users interact with the system.

Each screenshot included in the documentation should be properly labeled and explained. A short description is usually provided below each image to explain what the screenshot represents and what function it performs within the system.



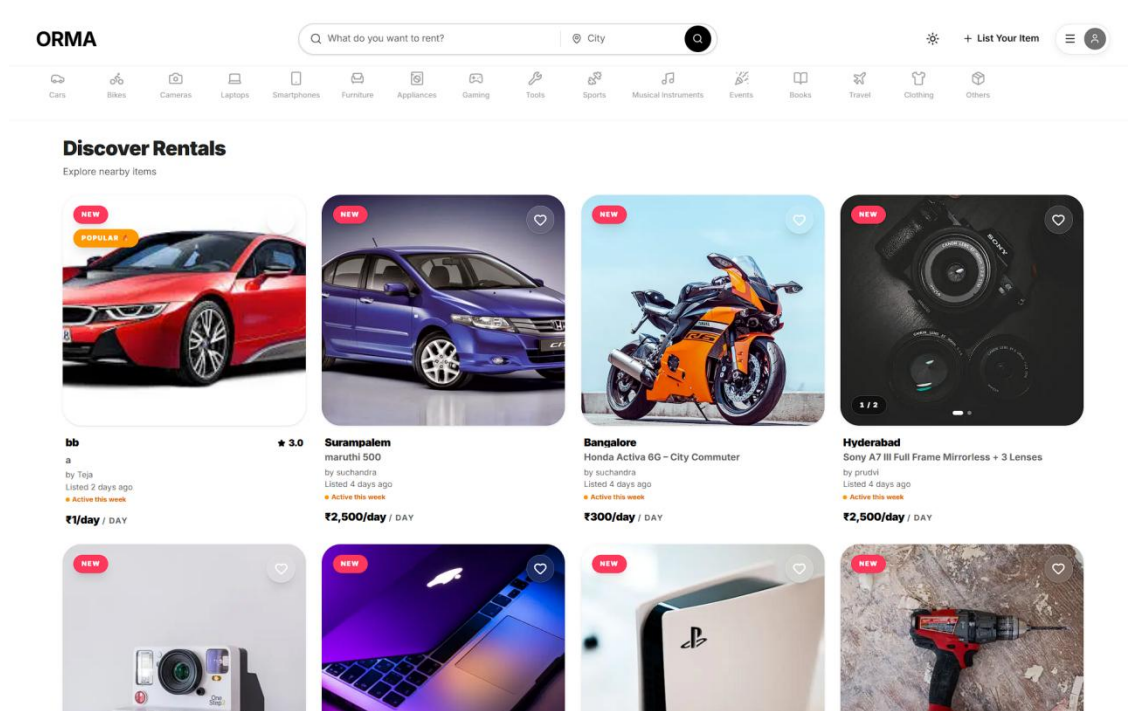
8.3 Home Page & Category Bar Screen

The home page and category bar screen is the first interface that users see when they access the ORMA platform. This screen is designed to provide an immediate overview of all available rental items and allow users to quickly filter by category. The home page serves as the primary entry point for the entire rental marketplace experience.

In the One-Stop Digital Rental Marketplace Platform, the home page displays a sticky navigation bar at the top containing the ORMA logo in coral red colour on the left, a rounded search pill in the centre with the placeholder text "Search anything to rent," and a user menu button on the right side. When a user is not logged in, the menu shows "Log in" and "Sign up" options. When logged in, it displays the user's avatar initial and a dropdown with links to Profile, My Listings, Wishlist, and Log out.

Directly below the navigation bar, a horizontal scrollable category bar displays icons and labels for all sixteen rental categories including Cars, Bikes, Cameras, Laptops, Mobile Phones, Furniture, Home Appliances, Gaming, Tools, Sports, Musical Instruments, Event Equipment, Books, Drones, Clothing, and Others. Each category icon is rendered using the Lucide React icon library. When a user clicks on a category, the active category receives a dark underline indicator and the listing grid below filters to show only items from that selected category.

The main content area displays rental listings in a responsive card grid layout. Each listing card shows an image with a heart-shaped wishlist icon in the top-right corner, the item location, title, category name, rental price per day in Indian Rupees format, and star rating if reviews exist. The grid adjusts automatically to show four columns on large desktop screens, three columns on tablets, two columns on small tablets, and a single column on mobile devices.



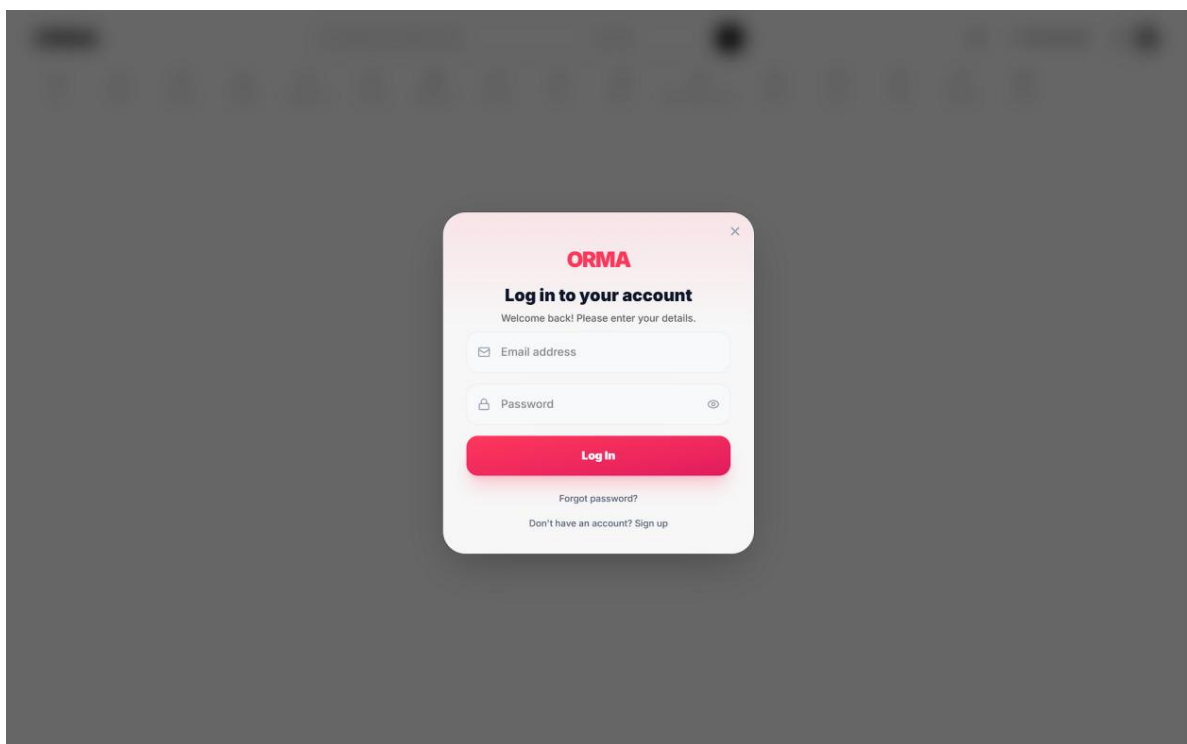
8.4 Authentication & Login Modal Screen

The authentication and login modal screen provides the user registration and login functionality for the ORMA platform. Unlike traditional login pages that navigate to a separate URL, ORMA uses a modal popup overlay that appears on top of the current page, matching the Airbnb-style authentication experience. This design allows users to log in without losing their current browsing context.

In the One-Stop Digital Rental Marketplace Platform, the authentication modal opens when a user clicks "Log in" or "Sign up" from the navigation bar dropdown menu, or when an unauthenticated user attempts to perform a protected action such as saving to wishlist, creating a listing, or contacting an owner. The modal appears with a semi-transparent backdrop blur effect and a centered white card with rounded corners.

The modal contains two tabs at the top: "Log in" and "Sign up." The Log in tab displays input fields for email address and password, a coral-red "Log In" button spanning the full width, a "Forgot password?" link, a horizontal divider with "or" text, and a "Continue with Google" button with the Google icon. The Sign up tab displays input fields for Full Name, Email, Phone Number (optional), Password, and Confirm Password, followed by a "Create Account" button and the same Google OAuth option.

The modal includes proper form validation that checks for valid email format, minimum password length, and matching password confirmation. Error messages are displayed below the relevant input fields in red text. When login or signup succeeds, the modal closes automatically, a success toast notification appears, and the navigation bar updates to show the user's avatar initial in the user menu button.



8.5 Listing Detail & Contact Owner Screen

The listing detail and contact owner screen is one of the most important interfaces in the system. This screen displays comprehensive information about a specific rental item and provides the primary mechanism for renters to connect with item owners directly.

When a user clicks on any listing card from the home page or search results, the system navigates to the listing detail page using a dynamic URL route. The page layout follows Airbnb's design pattern with a two-column structure on desktop: the left column (approximately sixty percent width) contains the main content, while the right column (approximately forty percent width) contains a sticky pricing and contact card.

The top section of the page displays an image gallery. When a listing has five or more images, they are displayed in a grid layout with one large image on the left and four smaller images arranged in a two-by-two grid on the right. Clicking any image opens a fullscreen gallery modal with navigation arrows and thumbnail strip. Below the gallery, the page shows the item title as a large heading, followed by the location, category, and condition information.

The owner information section displays the owner's avatar, name, verification badge if verified, member since date, average rating, and a short bio. Below this, the description section presents the full item description with a "Show more" expandable option for longer texts. The terms and conditions section lists details such as security deposit amount, ID proof requirement, delivery availability, and minimum rental period.

The screenshot shows the ORMA app interface for selecting a category. At the top, there's a header with the ORMA logo and navigation options: 'Anywhere', 'Any week', 'Add guests', and a red circular button with a plus sign. Below the header, it says 'Step 1 of 7 - Category'. The main section is titled 'What kind of item are you listing?' with a subtitle 'Select the category that best describes your item.' Below this, there's a grid of 16 category buttons, each with an icon and a label: Cars, Bikes, Cameras, Laptops, Smartphones, Furniture, Appliances, Gaming, Tools, Sports, Musical Instruments, Events, Books, Travel, Clothing, and Others. At the bottom, there are 'Back' and 'Continue -->' buttons.

8.6 Create Listing & Multi-Step Form Screen

The create listing screen provides the interface through which item owners can list their items for rent on the ORMA platform. This screen implements a multi-step form process inspired by Airbnb's hosting flow, guiding owners through seven sequential steps to create a complete and professional listing.

The create listing page is a protected route that requires user authentication. When an unauthenticated user attempts to access this page, the authentication modal opens automatically. Once authenticated, the user sees a clean interface with the ORMA logo on the left, the current step indicator in the centre showing "Step X of 7," and an exit button on the right. A coloured progress bar at the top shows the completion percentage.

Step one asks the owner to select the item category from a grid of category cards with icons and labels. Step two provides a drag-and-drop zone for uploading item photos, with a minimum of one and maximum of ten images allowed. Step three collects item details including title, description, brand, model, and condition. Step four allows the owner to set rental pricing for different periods including per hour, per day, per week, and per month, along with a security deposit amount. Step five collects the item location including city, area, state, and pin code. Step six gathers contact information and rental terms. Step seven displays a review of all entered information with edit buttons for each section and a prominent "Publish Listing" button.

Navigation between steps is handled by "Back" and "Next" buttons at the bottom of each step. The form data is managed using a Zustand state store that persists data across steps. When the user clicks "Publish Listing" on the final step, the system uploads images to Supabase Storage, inserts the listing data into the database, displays a success notification, and redirects the user to the newly created listing detail page.

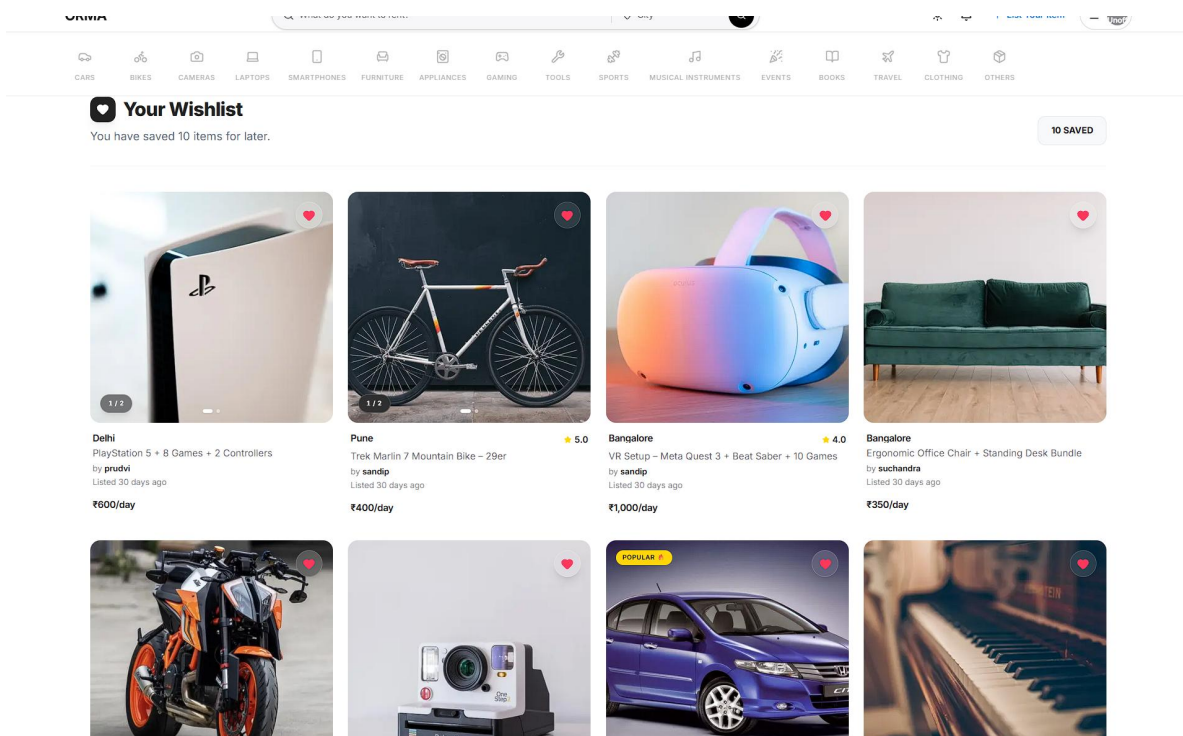
The screenshot shows the 'Add photos of your item' screen, which is Step 2 of 7 in the listing process. The interface features a top navigation bar with the ORMA logo, filters for 'Anywhere', 'Any week', and 'Add guests', and a '+ List Your Item' button. A progress indicator shows the current step. The main content area has a heading 'Add photos of your item' and a subtext 'Items with clear, well-lit photos get rented 3x faster. Add up to 10 photos.' Below this is a large dashed-border box containing a camera icon and the text 'Uploading...' and 'JPG, PNG, WebP - Max 5MB per photo'. At the bottom, there are 'Back' and 'Continue' buttons.

8.7 Search, Filter & Wishlist Screen

The search, filter, and wishlist screens are essential components of the One-Stop Digital Rental Marketplace Platform that enable users to discover specific rental items efficiently and save items for future reference. These screens demonstrate the platform's advanced browsing capabilities and personalization features.

When the user clicks the search bar in the navigation bar or navigates to the search page, the system displays an enhanced search interface with two input fields: "What are you looking for?" for the item search query and "Where?" for the city or location filter. Popular search terms such as "Camera," "Laptop," "Bike," and "PS5" are displayed as clickable chips below the search inputs. When the user submits a search, the system queries the Supabase database using full-text search on the listing title and description fields.

The search results page displays the number of results found along with the search context, such as "Showing 24 results for 'camera' in Hyderabad." Active filters appear as removable chips below the results count, such as "Cameras ✕," "Hyderabad ✕," and "₹500-₹2000 ✕," with a "Clear all filters" option. Sort options are displayed as horizontal pills including Recommended, Newest, Price Low to High, Price High to Low, and Top Rated. The filter panel provides options for category selection with item counts, price range inputs, city dropdown, condition checkboxes, and availability toggle.



9 CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The One-Stop Digital Rental Marketplace Platform (ORMA) is designed to provide a unified, transparent, and commission-free digital marketplace where anyone can list items for rent and anyone can find and contact owners directly. In today's digital world, the sharing economy is rapidly growing and people increasingly prefer renting over buying for temporary needs.

The main objective of this project is to address the lack of organized rental services in Tier-2 and Tier-3 cities by building a web application where local vendors and individual owners can list their idle items for rent, and users can browse, search, filter, and contact them directly through phone or WhatsApp. The system supports sixteen rental categories including Cars, Bikes, Cameras, Laptops, Mobile Phones, Furniture, Home Appliances, Gaming Consoles, Tools, Sports Equipment, Musical Instruments, Event Equipment, Books, Drones, Clothing, and miscellaneous items.

One of the major advantages of this system is that it operates on a completely commission-free model, meaning that neither owners nor renters pay any fees to the platform. All rental transactions happen directly between the two parties, similar to how OLX facilitates buying and selling. The platform serves purely as a discovery and connection layer, making it accessible for small local vendors who cannot afford commission-based platforms. This approach removes the financial barrier that prevents many local rental service providers from establishing a digital presence.

Overall, the One-Stop Digital Rental Marketplace Platform successfully achieves its objective of creating a unified, transparent, and user-friendly rental marketplace. The system works effectively across all device sizes and provides an interactive experience for both renters and owners. This project highlights the potential of combining modern web technologies with the sharing economy model to create accessible digital platforms that serve underserved markets and empower local businesses.

9.2 Future Scope

Although the One-Stop Digital Rental Marketplace Platform works effectively as a web application, there are several opportunities for improvement and future development. With advancements in web technologies, mobile development frameworks, and artificial intelligence, this system can be enhanced to provide even more comprehensive and personalized rental marketplace services.

One possible improvement is the development of a native mobile application for Android and iOS platforms. Currently, the system is designed as a responsive web application that works on mobile browsers. However, a dedicated mobile application could provide features such as push notifications for new inquiries, offline browsing of saved listings, camera integration for faster photo uploads during listing creation, and location-based services using GPS to show nearby rental items. Technologies such as React Native or Flutter could be used to build cross-platform mobile applications that share much of the existing business logic.

Another improvement could involve integrating a secure in-app payment system. While the current design intentionally avoids payment processing to keep the platform simple and commission-free, future versions could offer optional escrow-based payment services where the platform holds the rental fee until the item is returned, providing financial protection for both parties. This would increase trust and reduce the risk associated with high-value rentals such as cameras, laptops, and vehicles.

The system can also be enhanced by incorporating artificial intelligence and machine learning for smarter recommendations. Currently, listings are displayed based on recency and category filtering. In the future, the system could analyze user browsing patterns, search history, and wishlist behaviour to provide personalized item recommendations. Cosine similarity algorithms could suggest items similar to previously viewed listings, while collaborative filtering could recommend items popular among users with similar rental preferences.

Another future improvement could involve implementing a real-time chat system between renters and owners within the platform. While the current system facilitates contact through phone and WhatsApp, an in-app messaging feature would allow users to communicate without sharing personal phone numbers initially. This messaging system could include features such as read receipts, typing indicators, and the ability to share images of item condition during the rental negotiation process.

The verification system can also be strengthened in future versions. Currently, the platform supports basic profile verification. Future enhancements could include Aadhaar-based identity verification through DigiLocker API integration, automated document verification using OCR technology, and background check partnerships. A stronger verification ecosystem would significantly increase trust between users, particularly for high-value rentals.

The platform could also benefit from integrating map-based location services. Instead of text-based location filtering, users could view rental items on an interactive map, similar to Airbnb's map view feature. This would help users find items in their immediate neighbourhood and understand the distance they would need to travel for pickup. Integration with mapping APIs such as Google Maps or OpenStreetMap would enable this functionality.

In addition, the system could be expanded to include a vendor analytics dashboard with more sophisticated reporting features. Future versions could provide owners with insights such as demand trends by category and season, optimal pricing recommendations based on market analysis, competitor pricing comparisons, and revenue forecasting based on historical rental patterns. These analytics would help vendors make data-driven decisions to maximize their rental income.

The user interface can also be improved to provide a more interactive and engaging experience. Future versions of the system could include features such as video uploads for listings allowing owners to showcase items in motion, virtual try-on or preview capabilities for certain categories, augmented reality integration for furniture placement visualization, and voice-based search for hands-free item discovery.

Furthermore, the platform could introduce a rental insurance partnership where users can purchase short-term insurance coverage for rented items. This would protect both owners against item damage and renters against defective items. Integration with insurance providers could offer automated policy generation based on item value and rental duration.

The system could also support multi-language localization to serve users across different Indian states in their preferred language. Adding support for Hindi, Telugu, Tamil, Kannada, Marathi, Bengali, and other regional languages would significantly expand the platform's accessibility in Tier-2 and Tier-3 cities where English proficiency may be limited.

10. BIBLIOGRAPHY

Bibliography is an important section in project documentation because it lists all the sources of information that were referred to while developing the project. These sources may include books, research papers, websites, tutorials, and technical documentation. The bibliography helps acknowledge the original authors and resources that contributed to the understanding and development of the project.

During the development of the One-Stop Digital Rental Marketplace Platform (ORMA), various online resources and research materials were used to understand concepts such as web application development, digital marketplace architecture, database design, user authentication, responsive UI design, and sharing economy business models. These resources helped in gaining knowledge about modern frontend frameworks, Backend-as-a-Service platforms, and the implementation of scalable rental marketplace systems.

The bibliography section also helps readers and future researchers find additional information related to the project. By referring to these sources, readers can gain deeper knowledge about the technologies and methods used in the system.

The following subsections list the web resources and references that were used while designing and implementing the project.

10.1 Web Links

The following websites and online resources were referred to during the development of this project. These websites provide tutorials, documentation, and technical information related to web application development, database management, and digital marketplace systems.

<https://nextjs.org>

Official website and documentation for the Next.js framework used for building the frontend application with App Router, server-side rendering, and file-based routing.

<https://www.typescriptlang.org>

Official website of TypeScript programming language used for type-safe development across all application components.

<https://tailwindcss.com>

Official documentation for Tailwind CSS utility-first framework used for styling the entire user interface with responsive design.

<https://supabase.com/docs/guides/auth>

Supabase authentication documentation used for implementing email/password and Google OAuth login functionality.

<https://lucide.dev>

Official website for Lucide React icon library used for rendering category icons and UI elements throughout the application.

<https://www.framer.com/motion>

Documentation for Framer Motion animation library used for page transitions, card loading animations, and micro-interactions.

<https://github.com>

GitHub platform used for version control, code hosting, and accessing open-source projects related to rental marketplace development.

<https://vercel.com/docs>

Official documentation for Vercel hosting platform used for deploying the Next.js application with automatic SSL and CDN.

<https://react-hot-toast.com>

Documentation for React Hot Toast library used for displaying styled notification messages across the application.

<https://www.geeksforgeeks.org>

Educational website used for learning programming concepts, database design patterns, and web development techniques.

<https://developer.mozilla.org>

Mozilla Developer Network providing comprehensive web development documentation for HTML, CSS, JavaScript, and web APIs.

10.2 References

The following research papers, technical guides, and academic materials were referred to during the development of the One-Stop Digital Rental Marketplace Platform.

Fraiberger, S., and Sundararajan, A. (2016). Peer-to-Peer Rental Markets in the Sharing Economy. Harvard Business School Working Paper.

Hirao, M. (2023). Which Product Categories Welcome Peer-to-Peer Product Rental Market? SSRN Electronic Journal.

IJERT Research Team (2022). Peer to Peer Online Rental Platform. International Journal of Engineering Research and Technology.

Academic Research Paper (2024). A Study on Competitive Strategy Adopted by NoBroker. Alameen Educational Institution.

Codica Development Team (2025). How to Build a Rental Marketplace: Ultimate Guide. Codica Blog.

Suffescom Solutions (2025). How to Build a Rental Marketplace Platform: Cost and Steps. Suffescom Blog.

Mohammed Raffiyutheen, J. (2024). Rent Ease — A Modern Rental Platform. CSN-SECP Journal.

IJCRT Research Team (2021). A Survey on Online Renting Platforms. International Journal of Creative Research Thoughts.

Future Market Insights (2025). Online Home Rental Market Size, Trends and Forecast 2025. Market Research Report.

Pressman, R. S., and Maxim, B. R. (2014). Software Engineering: A Practitioner's Approach. McGraw-Hill Education.

Sommerville, I. (2016). Software Engineering. Pearson Education.

Elmasri, R., and Navathe, S. B. (2015). Fundamentals of Database Systems. Pearson Education.

Flanagan, D. (2020). JavaScript: The Definitive Guide. O'Reilly Media.

Next.js Documentation — React Framework for Production Web Applications.